Microsoft Cabinet Format

Page Options

The Cabinet Software Development Kit provides developers with the components needed to utilize Microsoft's Cabinet File technology within other applications, or to build cabinet file management tools. Microsoft is committed to making cabinet files an open technology.

This release of the Cabinet Software Development Kit (formerly called the Cabinet Resource Kit) provides complete documentation of cabinet files, including the LZX data compression technology.

Another tool, called CABView can be found in the Microsoft Windows 95 Power Toys Web site. CABView allows you to treat CAB files like folders. Using this tool, you can explore and drag-and-drop within and into CAB files as you would a folder in Windows Explorer.

Download the Cabinet SDK from the following location:
http://download.microsoft.com/download/platformsdk/cab/2.0/w98nt42kmexp/en-us/Cabsdk.exe

In this Library Section

Cabarc User's Guide

Cabinet Format

FCI / FDI

LZX Format

MakeCAB User's Guide

MSZip Format

Microsoft Cabarc User's Guide

Copyright © 1997 Microsoft Corporation. All rights reserved.

Topics in this section

Introduction

- The Cabinet Format
- Cabarc
- Command Line Usage

Creating Cabinets

- Wildcards
- Folders
- Path Name Preservation
- Path Stripping
- Recursive Directory Search
- Reserve Space for Code Signature
- Set Cabinet ID
- Set Compression Type
- File List from a File

List Cabinet Contents

Extracting Cabinets

Introduction

The Cabinet Format

The cabinet format provides a way to efficiently package multiple files. The key features of the cabinet format are that multiple files may be stored in a single cabinet ("CAB file"); and that data compression is performed across file boundaries, significantly improving the compression ratio.

Depending upon the number of files to be compressed, and the expected access patterns (sequential or random access; whether most of the files will be requested at once or only a small portion), cabinets can be constructed in different ways. One key concept of the cabinet file is the *folder*. A *folder* is a collection of one or more files that are compressed together as a single entity. By compressing files in this way, the compression ratio is improved. The downside is that random access time suffers, since in order for any particular file in a folder to be decoded, all preceding files in the same folder must also be decoded.
Back to top

Cabarc

Cabarc is a utility that creates, extracts, and lists the contents of cabinet files (CABs), using a command line interface similar to that of

popular archiving tools. Cabarc supports wildcards and recursive directory searches.

Back to: Top of page > Introduction

Command Line Usage

Cabarc is used as follows:

```
Usage: CABARC [options] command cabfile [@list] [files] [dest_dir]
```

Currently, only three commands are supported; N (create new cabinet), L (list contents of an existing cabinet), and X (extract files from a cabinet). These commands are described in the following pages.

Options must appear before the command name, and cannot be combined (for example, to set the –r and –p options, use –r –p, and not –rp).

Back to: Top of page > Introduction

Creating Cabinets

Cabinets are created using the *n* command, followed by the name of the cabinet to create, followed by a filename list, as shown below:

```
cabarc n mycab.cab prog.c prog.h prog.exe readme.txt
```

The above command creates the cabinet *mycab.cab* containing the files "prog.c", "prog.h", "prog.exe", and "readme.txt", in a single folder, using the default compression mode, MSZIP.

Back to: Top of page

Wildcards

Cabarc supports wildcards in the filename list, as shown in the example below:

```
cabarc n mycab.cab prog.* readme.txt
```

Back to: Top of page > Creating Cabinets

Folders

By default, all files are added to a single folder (compression history) in the cabinet. It is possible to tell cabarc to begin a new folder, by inserting the plus (+) symbol as a file to be added, as shown below:

```
cabarc n mycab.cab test.c main.c + test.exe *.obj
```

The above command creates the cabinet "mycab.cab" with one folder containing "test.c" and "main.c", and a second folder containing "test.exe" and all files matching "*.obj".

Back to: Top of page > Creating Cabinets

Path Name Preservation

By default, directory names are not preserved in the cabinet; only the filename component is stored. For example, the following command will result in the filename "prog.c" being stored in the cabinet:

```
cabarc n mycab.cab c:\source\myproj\prog.c
```

In order to preserve path names, the –p option should be used:

```
cabarc -p n mycab.cab c:\mysource\myproj\prog.c
```

This command will cause the file to be named "mysource\myproj\prog.c" in the cabinet. Note that the c:\ prefix is still stripped from the filename; cabarc will not allow absolute paths to be stored in the cabinet, nor will it extract such absolute paths.

Back to: Top of page > Creating Cabinets

Path Stripping

In many situations it may be desirable to preserve some of the path name, but not all of it. For example, one might wish to archive everything in the c:\mysource\myproj\ directory, but store only the myproj\ component of the path. This can be accomplished with the path stripping option, -P (capital P).

```
cabarc -p -P mysource\ n mycab.cab c:\mysource\myproj\prog.c
```

The –P option strips any strings which begin with the provided string (wildcards are not supported in this case; it is a simple text match). Any absolute path prefixes such as c:\ or \ are stripped before the comparison takes place, so these characters should not be included in the –P option.

The –P option may be used multiple times to strip out multiple paths; cabarc builds a list of all paths to be stripped, and applies only the first one which matches. For example:

```
cabarc -p -P mysrc\ -P yoursrc\ n mycab.cab c:\mysrc\myproj\*.* d:\yoursrc\yourproj\*.c
```

The trailing slash at the end of the path name is important; entering `-P mysrc` instead of `-P mysrc\` would cause files to be added as "\myproj\<filename>".

Back to: Top of page > Creating Cabinets

Recursive Directory Search

Cabarc can archive files in a directory and all of its subdirectories, by use of the –r option. For example, the command shown below will archive all files ending in .h that are in c:\msdev\include\, c:\msdev\include\sys, and c:\msdev\include\gl (assuming these directories exist on your system).

```
cabarc -r -p n mycab.cab c:\msdev\include\*.h
```

The –p option is used here to preserve the path information when the files are added to the cabinet; without this option, only the filename components would be stored, although sometimes it might be desirable behavior to not use –p.

Back to: Top of page > Creating Cabinets

Reserve Space for Code Signature

Cabarc can reserve space in the cabinet for a code signature. This is done using the –s option, which reserves a specified amount of empty space in the cabinet. For code signing, 6144 bytes need to be reserved:

```
cabarc -s 6144 n mycab.cab test.exe
```

Note that the –s option does not actually write the code signature; it merely reserves space for it in the cabinet. The appropriate code signing utility must be used to fill out the code signature.

Back to: Top of page > Creating Cabinets

Set Cabinet ID

Cabinet files have a 16-bit cabinet ID field that is designed for application use. The default value of this field is zero, however, the –i option of cabarc can be used to set this field to any 16-bit value:

```
cabarc -i 12345 n mycab.cab test.exe
```

Back to: Top of page > Creating Cabinets

Set Compression Type

The default compression type for a cabinet is MSZIP. However, the compression type can be changed with the –m option. Currently only MSZIP compression (-m MSZIP) and no compression (-m NONE) are supported.

The following command stores files in the cabinet with no compression:

Back to: Top of page > Creating Cabinets

```
cabarc -m NONE n mycab.c *.*
```

File List From a File

Cabarc can input its list of files from a text file, instead of from the command line, by using @files ("at files"). This is done by prefixing with the @ symbol the name of the file which contains the file list. For example:

```
cabarc n mycab.cab @filelist.txt
```

The text file must list the physical file names of the files to be added, one per line. As is the case when specifying filenames on the command line, the plus (+) symbol can be used as a filename to specify the beginning of a new folder. If a filename contains any embedded spaces, it must be enclosed as quotes, as shown below:

```
test.c

myapp.exe

"output file.exe"
```

The reason for requiring quotes is that each physical filename may be followed on the same line by an optional logical filename, which specifies the name under which the file will be stored in the cabinet:

```
test.c myapp.c

myapp.exe

"output file.exe" foobar.exe
```

If the logical filename contains spaces, then it must also be enclosed in quotes. Note that the logical filename overrides the –p (preserve path names) and –P (strip path name) options -the file will be added to the cabinet *exactly* as indicated. Wildcards are allowed in the physical filename, but in this situation a logical filename is not allowed.

The "@" feature may be used multiple times, to retrieve file lists from multiple files. Cabarc does not check for the presence of duplicate files, so if the same physical file appears in multiple file lists, it will be added to the cabinet multiple times.

The "@" feature may be combined with filenames on the command line. Files are added in the order in which they are parsed on the

command line. Example:

```
cabarc n mycab.cab @filelist1.txt *.c @filelist2.txt *.h
```

Note: The "@" feature is available only when creating cabinets, not when extracting or listing cabinets.

Back to: Top of page > Creating Cabinets

List Cabinet Contents

It is possible to view the contents of a cabinet using the L (list) command, as shown below:

```
cabarc l mycab.cab
```

Cabarc will display the Set ID in the cabinet (see the –s option for cabinet creation), as well as the name of each file in the cabinet, along with its file size, file date, file time, and file attributes.

Back to: Top of page

Extracting Cabinets

The X (extract) command extracts files from a cabinet. The simplest use of the X command is shown below, which causes all files to be extracted from the cabinet:

```
cabarc x mycab.cab
```

Alternatively, it is possible to selectively extract files, by providing a list of filenames and/or wildcards:

```
cabarc x mycab.cab readme.txt *.exe *.c
```

By default, full path names (if they are present in the cabinet) are *not* preserved upon extraction. For example, if a file named *mysrc\myproj\test.c* is present in the cabinet, then the command `cabarc x mycab.cab` will cause the file *test.c* to be extracted into the current directory. In order to preserve file names upon extraction, the –p option must be used. This option will cause any required directories to be created if necessary.

Only the filename component is considered in the matching process; the pathname is discounted. For example, `cabarc x mycab.cab test.c` will cause the file *mysrc\myproj\test.c* to be extracted to the current directory as *test.c*, as will `cabarc x mycab.cab *.c` (which will also extract any other files matching *.c).

By default, the extracted files are stored in the current directory (and its subdirectories, if –p is used). However, it is possible to specify a destination directory for the extracted files. This is accomplished by appending a directory name to the command line. The directory name must end in a backslash (\ ). Examples:

```
cabarc x mycab.cab c:\somedir\
```

```
cabarc x mycab.cab *.exe c:\somedir\
```

Back to: Top of page

Microsoft Cabinet File Format

Copyright © 1997 Microsoft Corporation. All rights reserved.

Topics in this section

Introduction

Specification

- Conventions
- Overview
- Detailed Structure Specification
  - CFHEADER
  - CFFOLDER
  - CFFILE

Sample Cabinet File

Notes

- Checksum Method
- UTF Encoding Method

Introduction

This specification defines the Microsoft cabinet file format. Cabinet files are compressed packages containing a number of related files. The format of a cabinet file is optimized for maximum compression. Cabinet files support a number of compression formats, including MSZIP, LZX, or uncompressed. This document does not define these internal compression formats. For data compression formats, refer to the documents titled *Microsoft MSZIP Data Compression Format* and *Microsoft LZX Data Compression Format*.

Back to: Top of page

Specification

This segment of the documentation includes the following topics:

Conventions

Overview

Detailed Structure Specification

- CFHEADER
- CFFOLDER
- CFFILE

Back to: Top of page

Conventions

The types `u1`, `u2`, and `u4` are used to represent unsigned 8-, 16-, and 32-bit integer values, respectively. All multi-byte quantities are stored in little-endian order, where the least significant byte comes first.

The cabinet file format is described here using a C-like structure notation, where successive fields appear in the structure sequentially without padding or alignment. Header fields followed by (optional) may or may not be present, depending on the values in the `CFHEADER` flags byte.

Back to: Top of page > Specification

Overview

Each file stored in a cabinet is stored completely within a single folder. A cabinet file may contain one or more folders, or portions of a folder. A folder can span across multiple cabinets. Such a series of cabinet files form a set. Each cabinet file contains name information for the logically adjacent cabinet files. Each folder contains one or more files. Throughout this discussion, cabinets are said to contain "files". This is for semantic purposes only. Cabinet files actually store streams of bytes, each with a name and some other common attributes. Whether these byte streams are actually files or some other kind of data is application-defined.

A cabinet file contains a cabinet header (`CFHEADER`), followed by one or more cabinet folder (`CFFOLDER`) entries, a series of one or more cabinet file (`CFFILE`) entries, and the actual compressed file data in `CFDATA` entries. The compressed file data in the `CFDATA` entry is stored in one of several compression formats, as indicated in the corresponding `CFFOLDER` structure. The compression encoding formats used are detailed in separate documents.

Back to: Top of page > Specification

Detailed Structure Specification

This segment of the documentation includes the following topics:

CFHEADER

CFFOLDER

CFFILE

CFDATA

Back to: Top of page > Specification

CFHEADER

The `CFHEADER` structure provides information about this cabinet file.

```
struct CFHEADER
{
  u1  signature[4]inet file signature */
  u4  reserved1     /* reserved */
  u4  cbCabinet    /* size of this cabinet file in bytes */
  u4  reserved2     /* reserved */
  u4  coffFiles/* offset of the first CFFILE entry */
  u4  reserved3     /* reserved */
  u1  versionMinor   /* cabinet file format version, minor */
  u1  versionMajor   /* cabinet file format version, major */
  u2  cFolders  /* number of CFFOLDER entries in this */
                    /*    cabinet */
  u2  cFiles      /* number of CFFILE entries in this cabinet */
  u2  flags        /* cabinet file option indicators */
  u2  setID        /* must be the same for all cabinets in a */
                    /*    set */
  u2  iCabinet;         /* number of this cabinet file in a set */
  u2  cbCFHeader;       /* (optional) size of per-cabinet reserved */
                    /*    area */
  u1  cbCFFolder;       /* (optional) size of per-folder reserved */
                    /*    area */
  u1  cbCFData;         /* (optional) size of per-datablock reserved */
                    /*    area */
  u1  abReserve[];      /* (optional) per-cabinet reserved area */
  u1  szCabinetPrev[];  /* (optional) name of previous cabinet file */
  u1  szDiskPrev[];     /* (optional) name of previous disk */
  u1  szCabinetNext[];  /* (optional) name of next cabinet file */
  u1  szDiskNext[];     /* (optional) name of next disk */
};
```

u1 signature[4]

Contains the characters 'M','S','C','F' (bytes 0x4D, 0x53, 0x43, 0x46). This field is used to assure that the file is a cabinet file.

Back to: Top of page > Specification > CFHEADER

u4 reserved1
Reserved field, set to zero.

Back to: Top of page > Specification > CFHEADER

u4 cbCabinet
Total size of this cabinet file in bytes.

Back to: Top of page > Specification > CFHEADER

u4 reserved2
Reserved field, set to zero.

Back to: Top of page > Specification > CFHEADER

u4 coffFiles
Absolute file offset of first `CFFILE` entry.

Back to: Top of page > Specification > CFHEADER

u4 reserved3
Reserved field, set to zero.

Back to: Top of page > Specification > CFHEADER

u1 versionMinor
u1 versionMajor
Cabinet file format version.
Currently, `versionMajor` = 1 and `versionMinor` = 3.

Back to: Top of page > Specification > CFHEADER

u2 cFolders
The number of `CFFOLDER` entries in this cabinet file.

Back to: Top of page > Specification > CFHEADER

u2 cFiles
The number of `CFFILE` entries in this cabinet file.

Back to: Top of page > Specification > CFHEADER

u2 flags
Bit-mapped values that indicate the presence of optional data:

```
#define cfhdrPREV_CABINET       0x0001
#define cfhdrNEXT_CABINET       0x0002
#define cfhdrRESERVE_PRESENT    0x0004
```

`flags.cfhdrPREV_CABINET` is set if this cabinet file is not the first in a set of cabinet files. When this bit is set, the `szCabinetPrev` and `szDiskPrev` fields are present in this `CFHEADER`.

`flags.cfhdrNEXT_CABINET` is set if this cabinet file is not the last in a set of cabinet files. When this bit is set, the `szCabinetNext` and `szDiskNext` fields are present in this `CFHEADER`.

`flags.cfhdrRESERVE_PRESENT` is set if this cabinet file contains any reserved fields. When this bit is set, the `cbCFHeader`, `cbCFFolder`, and `cbCFData` fields are present in this `CFHEADER`.

Other bit positions in the `flags` field are reserved.

Back to: Top of page > Specification > CFHEADER

u2 setID
An arbitrarily derived (random) value that binds a collection of linked cabinet files together. All cabinet files in a set will contain the same `setID`. This field is used by cabinet file extractors to assure that cabinet files are not inadvertently mixed. This value has no meaning in a cabinet file that is not in a set.

Back to: Top of page > Specification > CFHEADER

u2 iCabinet
Sequential number of this cabinet in a multi-cabinet set. The first cabinet has `iCabinet`=0. This field, along with `setID`, is used by cabinet file extractors to assure that this cabinet is the correct continuation cabinet when spanning cabinet files.

Back to: Top of page > Specification > CFHEADER

u2 cbCFHeader(optional)
If `flags.cfhdrRESERVE_PRESENT` is not set, this field is not present, and the value of `cbCFHeader` defaults to zero. Indicates the size in bytes of the `abReserve` field in this `CFHEADER`. Values for `cbCFHeader` range from 0 to 60,000.

Back to: Top of page > Specification > CFHEADER

u1 cbCFFolder(optional)
If `flags.cfhdrRESERVE_PRESENT` is not set, then this field is not present, and the value of `cbCFFolder` defaults to zero. Indicates the size in bytes of the `abReserve` field in each `CFFOLDER` entry. Values for `cbCFFolder` range from 0 to 255.

Back to: Top of page > Specification > CFHEADER

u1 cbCFData(optional)
If `flags.cfhdrRESERVE_PRESENT` is set, then this field is not present, and the value for `cbCFData` defaults to zero. Indicates the size in bytes of the `abReserve` field in each `CFDATA` entry. Values for `cbCFData` range from 0 to 255.

Back to: Top of page > Specification > CFHEADER

u1 abReserve[cbCFHeader](optional)

If `flags.cfhdrRESERVE_PRESENT` is set and `cbCFHeader` is non-zero, then this field contains per-cabinet-file application information. This field is defined by the application and used for application-defined purposes.

Back to: Top of page > Specification > CFHEADER

u1 szCabinetPrev[](optional)

If `flags.cfhdrPREV_CABINET` is not set, then this field is not present. NUL-terminated ASCII string containing the file name of the logically previous cabinet file. May contain up to 255 bytes plus the NUL byte. Note that this gives the name of the most-recently-preceding cabinet file that contains the initial instance of a file entry. This might not be the immediately previous cabinet file, when the most recent file spans multiple cabinet files. If searching in reverse for a specific file entry, or trying to extract a file that is reported to begin in the "previous cabinet", `szCabinetPrev` would give the name of the cabinet to examine.

Back to: Top of page > Specification > CFHEADER

u1 szDiskPrev[](optional)

If `flags.cfhdrPREV_CABINET` is not set, then this field is not present. NUL-terminated ASCII string containing a descriptive name for the media containing the file named in `szCabinetPrev`, such as the text on the diskette label. This string can be used when prompting the user to insert a diskette. May contain up to 255 bytes plus the NUL byte.

Back to: Top of page > Specification > CFHEADER

u1 szCabinetNext[](optional)

If `flags.cfhdrNEXT_CABINET` is not set, then this field is not present. NUL-terminated ASCII string containing the file name of the next cabinet file in a set. May contain up to 255 bytes plus the NUL byte. Files extending beyond the end of the current cabinet file are continued in the named cabinet file.

Back to: Top of page > Specification > CFHEADER

u1 szDiskNext[](optional)

If `flags.cfhdrNEXT_CABINET` is not set, then this field is not present. NUL-terminated ASCII string containing a descriptive name for the media containing the file named in `szCabinetNext`, such as the text on the diskette label. May contain up to 255 bytes plus the NUL byte. This string can be used when prompting the user to insert a diskette.

Back to: Top of page > Specification > CFHEADER

CFFOLDER

Each `CFFOLDER` structure contains information about one of the folders or partial folders stored in this cabinet file. The first `CFFOLDER` entry immediately follows the `CFHEADER` entry. `CFHEADER.cFolders` indicates how many `CFFOLDER` entries are present.

Folders may start in one cabinet, and continue on to one or more succeeding cabinets. When the cabinet file creator detects that a folder has been continued into another cabinet, it will complete that folder as soon as the current file has been completely compressed. Any additional files will be placed in the next folder. Generally, this means that a folder would span at most two cabinets, but if the file is large enough, it could span more than two cabinets.

`CFFOLDER` entries actually refer to folder fragments, not necessarily complete folders. A `CFFOLDER` structure is the beginning of a folder if the `iFolder` value in the first file referencing the folder does not indicate the folder is continued from the previous cabinet file.

The `typeCompress` field may vary from one folder to the next, unless the folder is continued from a previous cabinet file.

Back to: Top of page > Specification

```
struct CFFOLDER
{
  u4  coffCabStart;  /* offset of the first CFDATA block in this
                     /*    folder */
  u2  cCFData;       /* number of CFDATA blocks in this folder */
  u2  typeCompress;  /* compression type indicator */
  u1  abReserve[];   /* (optional) per-folder reserved area */
};
```

u4 coffCabStart

Absolute file offset of first `CFDATA` block for this folder.

Back to: Top of page > Specification > CFFOLDER

u2 cCFData

Number of CFDATA structures for this folder that are actually in this cabinet. A folder can continue into another cabinet and have more CFDATA blocks in that cabinet, and a folder may have started in a previous cabinet. This number represents only the CFDATA structures for this folder that are at least partially recorded in this cabinet.

Back to: Top of page > Specification > CFFOLDER

u2 typeCompress

Indicates the compression method used for all `CFDATA` entries in this folder. The valid values are defined in each compression format's specification.

Back to: Top of page > Specification > CFFOLDER

u1 abReserve[CFHEADER.cbCFFolder](optional)

If `CFHEADER.flags.cfhdrRESERVE_PRESENT` is set and `cbCFFolder` is non-zero, then this field contains per-folder application information. This field is defined by the application and used for application-defined purposes.

Back to: Top of page > Specification > CFFOLDER

CFFILE

Each `CFFILE` entry contains information about one of the files stored (or at least partially stored) in this cabinet. The first `CFFILE` entry in each

cabinet is found at absolute offset `CFHEADER.coffFiles`. `CFHEADER.cFiles` indicates how many of these entries are in the cabinet. The `CFFILE` entries in a cabinet are ordered by `iFolder` value, then by `uoffFolderStart`. Entries for files continued from the previous cabinet will be first, and entries for files continued to the next cabinet will be last.

Back to: Top of page > Specification

```
struct CFFILE
{
  u4  cbFile;           /* uncompressed size of this file in bytes */
  u4  uoffFolderStart;  /* uncompressed offset of this file in the folder */
  u2  iFolder;          /* index into the CFFOLDER area */
  u2  date;             /* date stamp for this file */
  u2  time;             /* time stamp for this file */
  u2  attribs;          /* attribute flags for this file */
  u1  szName[];         /* name of this file */
};
```

u4 cbFile

Uncompressed size of this file in bytes.

Back to: Top of page > Specification > CFFILE

u4 uoffFolderStart

Uncompressed byte offset of the start of this file's data. For the first file in each folder, this value will usually be zero. Subsequent files in the folder will have offsets that are typically the running sum of the `cbFile` values.

Back to: Top of page > Specification > CFFILE

u2 iFolder

Index of the folder containing this file's data. A value of zero indicates this is the first folder in this cabinet file. The special `iFolder` values `ifoldCONTINUED_FROM_PREV` and `ifoldCONTINUED_PREV_AND_NEXT` indicate that the folder index is actually zero, but that extraction of this file would have to begin with the cabinet named in `CFHEADER.szCabinetPrev`. The special `iFolder` values `ifoldCONTINUED_PREV_AND_NEXT` and `ifoldCONTINUED_TO_NEXT` indicate that the folder index is actually one less than `CFHEADER.cFolders`, and that extraction of this file will require continuation to the cabinet named in `CFHEADER.szCabinetNext`.

```
#define ifoldCONTINUED_FROM_PREV      (0xFFFD)
#define ifoldCONTINUED_TO_NEXT        (0xFFFE)
#define ifoldCONTINUED_PREV_AND_NEXT  (0xFFFF)
```

Back to: Top of page > Specification > CFFILE

u2 date

Date of this file, in the format `((year-1980) << 9)+(month << 5)+(day)`, where `month`={1..12} and `day`={1..31}. This "date" is typically considered the "last modified" date in local time, but the actual definition is application-defined.

Back to: Top of page > Specification > CFFILE

u2 time

Time of this file, in the format `(hour << 11)+(minute << 5)+(seconds/2)`, where `hour`={0..23}. This "time" is typically considered the "last modified" time in local time, but the actual definition is application-defined.

Back to: Top of page > Specification > CFFILE

u2 attribs

Attributes of this file; may be used in any combination:

```
#define  _A_RDONLY       (0x01)  /* file is read-only */
#define  _A_HIDDEN       (0x02)  /* file is hidden */
#define  _A_SYSTEM       (0x04)  /* file is a system file */
#define  _A_ARCH         (0x20)  /* file modified since last backup */
#define  _A_EXEC         (0x40)  /* run after extraction */
#define  _A_NAME_IS_UTF  (0x80)  /* szName[] contains UTF */
```

All other attribute bit values are reserved.

Back to: Top of page > Specification > CFFILE

char szName[]

NUL-terminated name of this file. Note that this string may include path separator characters. When `attribs._A_NAME_IS_UTF` is set, this string can be converted directly to Unicode, avoiding locale-specific dependencies. See "UTF Encoding" for more information. When `attribs._A_NAME_IS_UTF` is not set, this string is subject to interpretation depending on locale.

Back to: Top of page > Specification > CFFILE

CFDATA

Each `CFDATA` record describes some amount of compressed data. The first `CFDATA` entry for each folder is located using `CFFOLDER.coffCabStart`. Subsequent `CFDATA` records for this folder are contiguous.

Back to: Top of page > Specification

```
struct CFDATA
{
  u4  csum;          /* checksum of this CFDATA entry */
  u2  cbData;        /* number of compressed bytes in this block */
  u2  cbUncomp;      /* number of uncompressed bytes in this block */
  u1  abReserve[];   /* (optional) per-datablock reserved area */
  u1  ab[cbData];    /* compressed data bytes */
};
```

u4 csum

Checksum of this `CFDATA` structure, from `CFDATA.cbData` through `CFDATA.ab[cbData-1]`. See "Checksum Method" for more information. May be set to zero if the checksum is not supplied.

Back to: Top of page > Specification > CFDATA

u2 cbData

Number of bytes of compressed data in this CFDATA record. When cbUncomp is zero, this field indicates only the number of bytes that fit into this cabinet file.

Back to: Top of page > Specification > CFDATA

u2 cbUncomp

The uncompressed size of the data in this CFDATA entry. When this CFDATA entry is continued in the next cabinet file, cbUncomp will be zero, and cbUncomp in the first CFDATA entry in the next cabinet file will report the total uncompressed size of the data from both CFDATA blocks.

Back to: Top of page > Specification > CFDATA

u1 abReserve[CFHEADER.cbCFData](optional)

If CFHEADER.flags.cfhdrRESERVE_PRESENT is set and cbCFHeader is non-zero, then this field contains per-datablock application information. This field is defined by the application and used for application-defined purposes.

Back to: Top of page > Specification > CFDATA

u1 ab[cbData]

The compressed data bytes, compressed using the CFFOLDER.typeCompress method. When cbUncomp is zero, these data bytes must be combined with the data bytes from the next cabinet's first CFDATA entry before decompression.

When `CFFOLDER.typeCompress` indicates that the data is not compressed, this field contains the uncompressed data bytes. In this case, `cbData` and `cbUncomp` will be equal unless this CFDATA entry crosses a cabinet file boundary.

Back to: Top of page > Specification > CFDATA

A Sample Cabinet File

```
       0   1   2   3   4   5   6    7   8   9   A   B   C   D   E   F
000   4D  53  43  46  00  00  00 00-FD  00  00  00  00  00  00  00  MSCF
010   2C  00  00  00  00  00  00 00-03  01  01  00  02  00  00  00
020   22  06  00  00  5E  00  00 00-01  00  00  00  4D  00  00  00
030   00  00  00  00  00  00  6C 22-BA  59  20  00  68  65  6C  6C  hell
040   6F  2E  63  00  4A  00  00 00-4D  00  00  00  00  00  6C  22  o.c
050   E7  59  20  00  77  65  6C 63-6F  6D  65  2E  63  00  BD  5A  welcome.c
060   A6  30  97  00  97  00  23 69-6E  63  6C  75  64  65  20  3C  #include <
070   73  74  64  69  6F  2E  68 3E-0D  0A  0D  0A  76  6F  69  64  stdio.h>    void
080   20  6D  61  69  6E  28  76 6F-69  64  29  0D  0A  7B  0D  0A  main(void)  {
090   20  20  20  20  70  72  69 6E-74  66  28  22  48  65  6C  6C  printf("Hell
0A0   6F  2C  20  77  6F  72  6C 64-21  5C  6E  22  29  3B  0D  0A  o, world!\n");
0B0   7D  0D  0A  23  69  6E  63 6C-75  64  65  20  3C  73  74  64  }  #include <std
0C0   69  6F  2E  68  3E  0D  0A 0D-0A  76  6F  69  64  20  6D  61  io.h>    void ma
0D0   69  6E  28  76  6F  69  64 29-0D  0A  7B  0D  0A  20  20  20  in(void)  {
0E0   20  70  72  69  6E  74  66 28-22  57  65  6C  63  6F  6D  65  printf("Welcome
0F0   21  5C  6E  22  29  3B  0D 0A-7D  0D  0A  0D  0A              !\n");  }
```

This is a very simple example of a cabinet file which contains two small text files, stored uncompressed for clarity.

Back to: Top of page

```
Offset   Description
00..23   CFHEADER
00..03   signature = 0x4D, 0x53, 0x43, 0x46
04..07   reserved1
08..0B   cbCabinet = 0x000000FD (253)
0C..0F   reserved2
10..13   coffFiles = 0x0000002C
14..17   reserved3
18..19   versionMinor, Major = 1.3
1A..1B   cFolders = 1
1C..1D   cFiles = 2
1E..1F   flags = 0 (no reserve, no previous or next cabinet)
20..21   setID = 0x0622
22..23   iCabinet = 0

24..2B   CFFOLDER[0]
24..27   coffCabStart = 0x0000005E
28..29   cCFData = 1
2A..2B   typeCompress = 0 (none)

2C..43   CFFILE[0]
2C..2F   cbFile = 0x0000004D (77 bytes)
30..33   uoffFolderStart = 0x00000000
34..35   iFolder = 0
36..37   date = 0x226C = 0010001 0011 01100 = March 12, 1997
38..39   time = 0x59BA = 01011 001101 11010 = 11:13:52 AM
3A..3B   attribs = 0x0020 = _A_ARCHIVE
3C..43   szName = "hello.c" + NUL

44..5D   CFFILE[1]
44..47   cbFile = 0x0000004A (74 bytes)
48..4B   uoffFolderStart = 0x0000004D
4C..4D   iFolder = 0
4E..4F   date = 0x226C = 0010001 0011 01100 = March 12, 1997
50..51   time = 0x59E7 = 01011 001111 00111 = 11:15:14 AM
52..53   attribs = 0x0020 = _A_ARCHIVE
54..5D   szName = "welcome.c" + NUL

5E..FD   CFDATA[0]
5E..61   csum = 0x30A65ABD
62..63   cbData = 0x0097 (151 bytes)
64..65   cbUncomp = 0x0097 (151 bytes)
66..FD   ab[0x0097] = uncompressed file data
```

Notes

Checksum Method

The computation and verification of checksums found in `CFDATA` entries cabinet files is done using a function named `CSUMCompute`. Its actual source code is provided for reference. When checksums are not supplied by the cabinet file creating application, the checksum field is set to

zero. Cabinet extracting applications do not compute or verify the checksum if the field is set to zero.

```
CHECKSUM CSUMCompute(void *pv, UINT cb, CHECKSUM seed)
{
    int         cUlong;                 // Number of ULONGs in block
    CHECKSUM    csum;                   // Checksum accumulator
    BYTE       *pb;
    ULONG       ul;

    cUlong = cb / 4;                    // Number of ULONGs
    csum = seed;                        // Init checksum
    pb = pv;                            // Start at front of data block

    //** Checksum integral multiple of ULONGs
    while (cUlong-- > 0) {
        //** NOTE: Build ULONG in big/little-endian independent manner
        ul = *pb++;                     // Get low-order byte
        ul |= (((ULONG)(*pb++)) <<  8); // Add 2nd byte
        ul |= (((ULONG)(*pb++)) << 16); // Add 3nd byte
        ul |= (((ULONG)(*pb++)) << 24); // Add 4th byte

        csum ^= ul;                     // Update checksum
    }

    //** Checksum remainder bytes
    ul = 0;
    switch (cb % 4) {
        case 3:
            ul |= (((ULONG)(*pb++)) << 16); // Add 3nd byte
        case 2:
            ul |= (((ULONG)(*pb++)) <<  8); // Add 2nd byte
        case 1:
            ul |= *pb++;                    // Get low-order byte
        default:
            break;
    }
    csum ^= ul;                         // Update checksum

    //** Return computed checksum
    return csum;
}
```

The checksums for non-split `CFDATA` blocks are computed first on the compressed data bytes, then on the `CFDATA` header area, starting at the `CFDATA.cbData` field:

```
CFDATA.cbData = cbCompressed;
CFDATA.cbUncomp = cbUncompressed;
csumPartial = CSUMCompute(&CFDATA.ab[0],CFDATA.cbData,0);
CFDATA.csum = CSUMCompute(&CFDATA.cbData,sizeof(CFDATA) -
sizeof(CFDATA.csum),csumPartial);
```

When blocks are split across cabinet file boundaries, the checksum for the partial block at the end of a cabinet file is computed first on the partial field of compressed data bytes, then on the header:

```
CFDATA.cbData = cbPartialData;
CFDATA.cbUncomp = 0;
csumPartial = CSUMCompute(&CFDATA.ab[0],cbPartialData,0);
CFDATA.csum = CSUMCompute(&CFDATA.cbData,sizeof(CFDATA) -
sizeof(CFDATA.csum),csumPartial);
```

The checksum for the residual block in the next cabinet file is computed first on the remainder of the field of compressed data bytes, then on the header:

```
CFDATA.cbData = cbResidualData;
CFDATA.cbUncomp = cbUncompressed;
csumPartial = CSUMCompute(&CFDATA.ab[cbPartialData],cbResidualData,0);
CFDATA.csum = CSUMCompute(&CFDATA.cbData,sizeof(CFDATA) -
sizeof(CFDATA.csum),csumPartial);
```

UTF Encoding Method

UTF (universal text format) is used to compactly represent a broad range of Unicode characters while favoring size for the most common characters. Unicode characters are translated to sequences of one, two, or three bytes per character.

When a string containing Unicode characters larger than 0x007F are encoded in the `CFFILE.szName` field, the `_A_NAME_IS_UTF` attribute should be included in the file's attributes. When no characters larger than 0x007F are in the name, the `_A_NAME_IS_UTF` attribute should not be set. If byte values larger than 0x7F are found in `CFFILE.szName`, but the `_A_NAME_IS_UTF` attribute is not set, the characters should be interpreted according to the current locale.

Unicode characters with values 0x0000 through 0x007F are represented by a single byte of the same value.

The first byte emitted for Unicode characters 0x0080 through 0x07FF is `0xC0+(unicodevalue >> 6)`, and the second byte is `0x80+(unicodevalue & 0x003F)`.

Unicode characters 0x0800 through 0xFFFF are represented by byte1 = `0xE0+(unicodevalue >> 12)`, byte2 = `0x80+((unicodevalue >> 6) & 0x3F)`, and byte3 = `0x80+(unicodevalue & 0x3F)`.

Microsoft FCI/FDI Library Description

Copyright © 1996-1997 Microsoft Corporation. All rights reserved.

Topics in this section

Introduction

FCI

- FCICreate
- FCIAddFile
- FCIFlushCabinet
- FCIFlushFolder
- FCIDestroy

FDI

- FDICreate
- FDIIsCabinet
- FDICopy
- FDIDestroy

Introduction

The FCI (File Compression Interface) and FDI (File Decompression Interface) libraries provide the ability to create and extract files from cabinets (also known as "CAB files"). In addition, the libraries provide compression and decompression capability to reduce the size of file data stored in cabinets.

The FCI and FDI libraries, FCI.LIB and FDI.LIB, are available in both 32-bit and 16-bit forms. However, the 16-bit version will run more slowly than the 32-bit version.

FCI and FDI support multiple simultaneous contexts, so it is possible to create or extract multiple cabinets simultaneously within the same application. If the application is multi-threaded, it is also possible to run a different context in each thread; however, it is not permitted for the application to use the *same* context simultaneously in multiple threads (e.g. one cannot call FCIAddFile from two different threads, using the same FCI context).

FCI and FDI operate using the technique of function callbacks; some of the parameters of the FCI and FDI APIs are pointers to functions in the client application. The parameters and purpose of these functions are explained fully in this document. The *fci_int.h* and *fd_int.h* header files provide macros for declaring the callback functions, and use keywords such HUGE, FAR, and DIAMONDAPI, which ensure that the functions are properly defined for both 32-bit and 16-bit operation. For example, in the case of the memory allocation and memory free functions, the following definitions exist in *fci_int.h*:

```
#define FNFCIALLOC(fn) void HUGE * FAR DIAMONDAPI fn(ULONG cb)
#define FNFCIFREE(fn) void FAR DIAMONDAPI fn(void HUGE *pv)
```

These declarations can be used as follows:

```
FNFCIALLOC(mem_alloc)
{
      return malloc(cb);
}

FNFCIFREE(mem_free)
{
      return free(memory);
}

some_function()
{
      hfci = FCICreate(
            &erf,
            filedest,
            mem_alloc,
            mem_free,
            etc.
      );
}
```

It should be noted that the FCI callback function names all begin with the string "FCI". In addition, the FCI and FDI i/o functions (open, close, read, write, seek) take different parameters, and cannot be used interchangeably.

The FDI i/o functions take parameters which are identical to those of the C run-time library routines _open, close, read, write, and lseek. The FCI i/o functions take similar parameters, with the addition of an error pointer in which to return an i/o error, and the client's context pointer originally passed in to the FCICreate API.

Two example applications are provided; *testfci* and *testfdi*. These applications demonstrate how all of the FCI and FDI APIs, respectively, may be used.

Back to: Top of page

FCI

The five FCI (File Compression Interface) APIs are:

API	Description
FCICreate	Create an FCI context
FCIAddFile	Add a file to the cabinet under construction
FCIFlushCabinet	Complete the current cabinet
FCIFlushFolder	Complete the current folder and start a new folder
FCIDestroy	Destroy an FCI context

Back to: Top of page

FCICreate

Back to: Top of page

```
HFCI DIAMONDAPI FCICreate(
      PERF               perf,
      PFNFCIFILEPLACED   pfnfiledest,
      PFNFCIALLOC        pfnalloc,
      PFNFCIFREE         pfnfree,
      PFNFCIOPEN         pfnopen,
      PFNFCIREAD         pfnread,
      PFNFCIWRITE        pfnwrite,
      PFNFCICLOSE        pfnclose,
      PFNFCISEEK         pfnseek,
      PFNFCIDELETE       pfndelete,
      PFNFCIGETTEMPFILE  pfnfcigtf,
      PCCAB              pccab,
      void FAR *         pv
);
```

Back to: Top of page > FCI > FCICreate

Parameters

perf

Pointer to an error structure

pfnfiledest

Function to call when a file is placed

pfnalloc

Memory allocation function

pfnfree

Memory free function

pfnopen

Function to open a file

pfnread

Function to read data from a file

pfnwrite

Function to write data to a file

pfnclose

Function to close a file

pfnseek

Function to seek to a new position in a file

pfntemp

Function to obtain a temporary file name

pfndelete

Function to delete a file

pccab

Parameters for creating cabinet

pv

Client context parameter

Back to: Top of page > FCI > FCICreate

Description

The **FCICreate** API creates an FCI context that is passed to other FCI APIs.

The *perf* parameter should point to a global or allocated ERF structure. Any errors returned by **FCICreate** or subsequent FCI APIs using the same context will cause the ERF structure to be filled out.

The *pfnalloc* and *pfnfree* parameters should point to memory allocation and memory free functions which will be called by FCI to allocate and free memory. These two functions take parameters identical to the standard C malloc and free functions.

The *pfnopen*, *pfnread*, *pfnwrite*, *pfnclose*, *pfnseek*, and *pfndelete* parameters should point to functions which perform file open, file read, file write, file close, file seek, and file delete operations respectively. These functions must accept parameters similar to those for the standard *_open*, *_read*, *_write*, *_close*, *_lseek*, and *remove* functions, with the addition of two additional parameters to the list; err and pv. The err parameter is an *int* *, and upon entry into the function, **err* will equal zero. However, if the function returns failure, **err* should be set to an

error code of the application's choosing, which will be returned via *perf* (the error code is not used by FCI, and is not required to conform to C run-time library *errno* conventions). The pv parameter will equal the client's context parameter passed in to **FCICreate**.

The *pfntemp* parameter should point to a function which returns the name of a suitable temporary file. Three parameters will be passed to this function; pszTempName, an area of memory to store the filename, cbTempName, the size of the memory area, and pv, the client's context pointer. The filename returned by this function should not occupy more than cbTempName bytes. FCI may open several temporary files at once, so it is important to ensure that a different filename is returned each time, and that the file does not already exist. The function should return TRUE for success, or FALSE for failure.

The *pfnfiledest* parameter should point to a function which will be called whenever the location of a file or file segment on a particular cabinet has been finalized. This information is useful only when files are being stored across multiple cabinets. The parameters passed to this function are pccab, a pointer to the CCAB structure of the cabinet on which the file has been stored, pszFile, the filename of the file which has been placed, cbFile, the file size, and fContinuation, a Boolean which signifies whether the file is a later segment of a file which has been split across cabinets. In addition, the client context value, pv, is also passed as a parameter.

The *pccab* parameter should point to an initialized CCAB structure, which will provide FCI with details on how to build the cabinet. The CCAB fields are explained below:

The cb field, the media size, specifies the maximum size of a cabinet which will be created by FCI. If necessary, multiple cabinets will be created. To ensure that only one cabinet is created, a sufficiently large number should be used for this parameter.

The cbFolderThresh field specifies the maximum number of compressed bytes which may reside in a folder before a new folder is created. A higher folder threshold improves compression performance (since creating a new folder resets the compression history), but increases random access time to the folder.

The iCab field is used by FCI to count the number of cabinets that have been created so far. This value can also be read by the application to determine the name of a cabinet. See the *GetNextCab* parameter of the **FCIAddFile** API for details.

The iDisk field is used in a similar manner to iCab. See the *GetNextCab* parameter of the **FCIAddFile** API for details.

The setID field is for the use of the application, and can be initialized with any number. The set ID is stored in the cabinet.

The szDisk field should contain a disk-specific string (such as "Disk1", "Disk2", etc.) corresponding to the disk on which the cabinet is placed. Alternatively, if cabinets are not spanning multiple disks, the string can simply be a null string. This field is stored in the cabinet and is used upon extraction to prompt the user to insert the correct disk. See the **FCIAddFile** API for details.

The szCab field should contain a string which contains the name of the first cabinet to be created (e.g. "APP1.CAB"). In the event of multiple cabinets being created, the *GetNextCab* function called by the **FCIAddFile** API allows subsequent cabinet names to be specified.

The szCabPath field should contain the complete path of where to create the cabinet (e.g. "C:\MYFILES\").

The `cbReserveCFHeader`, `cbReserveCFFolder`, and `cbReserveCFData` fields can be set to create per-cabinet, per-folder, and per-datablock reserved sections in the cabinet. For example, setting `cbReserveCFHeader` to 6144 is commonly used to reserve a 6k space in the cabinet file as needed for codesigning. The other reserved sections are not commonly used.

Back to: Top of page > FCI > FCICreate

Returns

If successful, a non-NULL HFCI context pointer is returned. If unsuccessful, NULL is returned, and the error structure pointed to by *perf* is filled out.

Back to: Top of page > FCI > FCICreate

FCIAddFile

Back to: Top of page

```
BOOL DIAMONDAPI FCIAddFile(
      HFCI                 hfci,
      char                *pszSourceFile,
      char                *pszFileName,
      BOOL                 fExecute,
      PFNFCIGETNEXTCABINET GetNextCab,
      PFNFCISTATUS         pfnProgress,
      PFNFCIGETOPENINFO    pfnOpenInfo,
      TCOMP                typeCompress
);
```

Back to: Top of page > FCI > FCIAddFile

Parameters

hfci

FCI Context pointer originally returned by **FCICreate**

pszSourceFile

Name of file to add (should include path information)

pszFileName

Name under which to store the file in the cabinet
fExecute

Boolean indicating whether the file should be executed when it is extracted

GetNextCab

Function called to obtain specifications on the next cabinet to create

pfnProgress

Progress function called to update the user

pfnOpenInfo

Function called to open a file and return file date, time and attributes

typeCompress

Compression type to use

Back to: Top of page > FCI > FCIAddFile

Description

The **FCIAddFile** API adds a file to the cabinet under construction.

The *hfci* parameter must be the context pointer returned by a previous call to **FCICreate**.

The *pszSourceFile* parameter specifies the location of the file to be added to the cabinet, and should therefore include as much path information as possible (e.g. "C:\MYFILES\TEST.EXE").

The *pszFileName* parameter specifies the name of the file inside the cabinet, and should not include any path information (e.g. "TEST.EXE").

The *fExecute* parameter specifies whether the file should be executed automatically when the cabinet is extracted. When set, the _A_EXEC attribute will be added to the file entry in the CAB. This mechanism is used in some Microsoft self-extracting executables, and could be used for this purpose in any custom extract application.

The *GetNextCab* parameter should point to a function which is called whenever FCI wishes to create a new cabinet, which will happen whenever the size of the cabinet is about to exceed the media size as specified in the cb field of the CCAB structure passed to **FCICreate**. The *GetNextCab* function is called with three parameters which are explained below:

The first parameter, pccab, is a pointer to a copy of the CCAB structure of the cabinet which has just been completed. However, the iCab field will have been incremented by one. When this function returns, the next cabinet will be created using the fields in this structure, so these fields should be modified as is necessary. In particular, the szCab field (the cabinet name) should be changed. If creating multiple cabinets, typically the iCab field is used to create the name; for example, the *GetNextCab* function might include a line that does:

```
sprintf(pccab->szCab, "FOO%d.CAB", pccab->iCab);
```

Similarly, the disk name, media size, folder threshold, etc. parameters may also be modified.

The second parameter, cbPrevCab, is an estimate of the size of the cabinet which has just been completed.

The last parameter, pv, is the application-defined value originally passed to **FCICreate**.

The *GetNextCab* function should return TRUE for success, or FALSE to abort cabinet creation.

The *pfnProgress* parameter should point to a function that is called periodically by FCI so that the application may send a progress report to the user. The progress function has four parameters; typeStatus, which specifies the type of status message, cb1 and cb2, which are numbers, the meaning of which is dependent upon typeStatus, and pv, the application-specific context pointer.

The typeStatus parameter may take on values of ***statusFile***, ***statusFolder***, or ***statusCabinet***. If typeStatus equals ***statusFile*** then it means that FCI is compressing data blocks into a folder. In this case, cb1 is either zero, or the compressed size of the most recently compressed block, and cb2 is either zero, or the uncompressed size of the most recently read block (which is usually 32K, except for the last block in a folder, which may be smaller). There is *no* direct relation between cb1 and cb2; FCI may read several blocks of uncompressed data before emitting any compressed data; if this happens, some ***statusFile*** messages may contain, for example, cb1 = 0 and cb2 = 32K, followed later by other messages which contain cb1 = 20K and cb2 = 0.

If typeStatus equals ***statusFolder*** then it means that FCI is copying a folder to a cabinet, and cb1 is the amount copied so far, and cb2 is the total size of the folder. Finally, if typeStatus equals ***statusCabinet***, then it means that FCI is writing out a completed cabinet, and cb1 is the estimated cabinet size that was previously passed to *GetNextCab*, and cb2 is the actual resulting cabinet size.

The progress function should return 0 for success, or -1 for failure, with an exception in the case of ***statusCabinet*** messages, where the function should return the desired cabinet size (*cb2*), or possibly a value rounded up to slightly higher than that.

The *pfnOpenInfo* parameter should point to a function which opens a file and returns its datestamp, timestamp, and attributes. The function will receive five parameters; pszName, the complete pathname of the file to open; pdate, a memory location to return a FAT-style date code; ptime, a memory location to return a FAT-style time code; pattribs, a memory location to return FAT-style attributes; and pv, the application-specific context pointer originally passed to **FCICreate**. The function should open the file using a file open function compatible with those passed in to **FCICreate**, and return the resulting file handle, or -1 if unsuccessful.

The *typeCompress* parameter specifies the type of compression to use, which may be either ***tcompTYPE_NONE*** for no compression, or ***tcompTYPE_MSZIP*** for Microsoft ZIP compression. Other compression formats may be supported in the future.

Back to: Top of page > FCI > FCIAddFile

Returns

If successful, TRUE is returned. If unsuccessful, FALSE is returned, and the error structure pointed to by *perf* (from **FCICreate**) is filled out.

Back to: Top of page > FCI > FCIAddFile

FCIFlushCabinet

Back to: Top of page

```
BOOL DIAMONDAPI FCIFlushCabinet(
      HFCI                  hfci,
      BOOL                  fGetNextCab,
      PFNFCIGETNEXTCABINET  GetNextCab,
      PFNFCISTATUS          pfnProgress
);
```

Back to: Top of page > FCI > FCIFlushCabinet

```
Parameters
```

hfci

FCI Context pointer originally returned by **FCICreate**

fGetNextCab

Name of file to add (should include path information)

GetNextCab

Function called to obtain specifications on the next cabinet to create

pfnProgress

Progress function called to update the user

Back to: Top of page > FCI > FCIFlushCabinet

Description

The **FCIFlushCabinet** API forces the current cabinet under construction to be completed immediately and written to disk. Further calls to **FCIAddFile** will cause files to be added to another cabinet. It is also possible that there exists pending data in FCI's internal buffers that will may require spillover into another cabinet, if the current cabinet has reached the application-specified media size limit.

The *hfci* parameter must be the context pointer returned by a previous call to **FCICreate**.

The *fGetNextCab* flag determines whether the function pointed to by the supplied *GetNextCab* parameter, will be called. If *fGetNextCab* is TRUE, then *GetNextCab* will be called to obtain continuation information. Otherwise, if *fGetNextCab* is FALSE, then *GetNextCab* will be called only if the cabinet overflows.

The *pfnProgress* parameter should point to a function which is called periodically by FCI so that the application may send a progress report to the user. This function works in an identical manner to the progress function passed to **FCIAddFile**.

Back to: Top of page > FCI > FCIFlushCabinet

Returns

If successful, TRUE is returned. If unsuccessful, FALSE is returned, and the error structure pointed to by *perf* (from **FCICreate**) is filled out.

Back to: Top of page > FCI > FCIFlushCabinet

FCIFlushFolder

Back to: Top of page

```
BOOL DIAMONDAPI FCIFlushFolder(
      HFCI                  hfci,
      PFNFCIGETNEXTCABINET  GetNextCab,
      PFNFCISTATUS          pfnProgress
);
```

Back to: Top of page > FCI > FCIFlushFolder

Parameters

hfci

FCI Context pointer originally returned by **FCICreate**

GetNextCab

Function called to obtain specifications on the next cabinet to create

pfnProgress

Progress function called to update the user

Back to: Top of page > FCI > FCIFlushFolder

Description

The **FCIFlushFolder** API forces the current folder under construction to be completed immediately, effectively resetting the compression history at this point (if compression is being used).

The *hfci* parameter must be the context pointer returned by a previous call to **FCICreate**.

The supplied *GetNextCab* function will be called if the cabinet overflows, which is a possibility if the pending data buffered inside FCI causes the application-specified cabinet media size to be exceeded.

The *pfnProgress* parameter should point to a function which is called periodically by FCI so that the application may send a progress report to the user. This function works in an identical manner to the progress function passed to **FCIAddFile**.

Back to: Top of page > FCI > FCIFlushFolder

FCIDestroy

Back to: Top of page

```
BOOL DIAMONDAPI FCIDestroy(
      HFCI  hfci
);
```

Back to: Top of page > FCI > FCIDestroy

Parameters

hfci

FCI Context pointer originally returned by **FCICreate**

Back to: Top of page > FCI > FCIDestroy

Description

The **FCIDestroy** API destroys an *hfci* context, freeing any memory and temporary files associated with the context.

Back to: Top of page > FCI > FCIDestroy

Returns

If successful, TRUE is returned. If unsuccessful, FALSE is returned. The only reason for failure is that the *hfci* passed in was not a proper context handle.

Back to: Top of page > FCI > FCIDestroy

FDI

The five FDI (File Decompression Interface) APIs are:

API	Description
FDICreate	**Create an FCI context**
FDIIsCabinet	**Determines whether a file is a cabinet, and returns information if so**
FDICopy	**Extracts files from cabinets**
FDIDestroy	**Destroy an FDI context**

Back to: Top of page

FDICreate

Back to: Top of page

```
HFCI DIAMONDAPI FDICreate(
      PFNALLOC  pfnalloc,
      PFNFREE   pfnfree,
      PFNOPEN   pfnopen,
      PFNREAD   pfnread,
      PFNWRITE  pfnwrite,
      PFNCLOSE  pfnclose,
      PFNSEEK   pfnseek,
      int       cpuType,
      PERF      perf
);
```

Back to: Top of page > FDI

Parameters

pfnalloc

Memory allocation function

pfnfree

Memory free function

pfnopen

Function to open a file

pfnread

Function to read data from a file

pfnwrite

Function to write data to a file

pfnclose

Function to close a file

pfnseek

Function to seek to a new position in a file

cpuType

Type of CPU

perf

Pointer to an error structure

Back to: Top of page > FDI > FDICreate

Description

The **FDICreate** API creates an FDI context that is passed to other FDI APIs.

The *pfnalloc* and *pfnfree* parameters should point to memory allocation and memory free functions which will be called by FDI to allocate and free memory. These two functions take parameters identical to the standard C malloc and free functions.

The *pfnopen*, *pfnread*, *pfnwrite*, *pfnclose*, and *pfnseek* parameters should point to functions which perform file open, file read, file write, file close, and file seek operations respectively. These functions should accept parameters identical to those for the standard *_open*, *_read*, *_write*, *_close*, and *_lseek* functions, and should likewise have identical return codes. Note that the FDI i/o functions do not take the same parameters as the FCI i/o functions.

It is not necessary for these functions to actually call _open etc.; these functions could instead call fopen, fread, fwrite, fclose, and fseek, or CreateFile, ReadFile, WriteFile, CloseHandle, and SetFilePointer, etc. However, the parameters and return codes will have to be translated appropriately (e.g. the file open mode passed in to *pfnopen*).

The *cpuType* parameter should equal one of ***cpu80386*** (indicating that 80386 instructions may be used), ***cpu80286*** (indicating that only 80286 instructions may be used), or ***cpuUNKNOWN*** (indicating that FDI should determine the CPU type). The *cpuType* parameter is looked at only by the 16-bit version of FDI; it is ignored by the 32-bit version of FDI.

The *perf* parameter should point to a global or allocated ERF structure. Any errors returned by **FDICreate** or subsequent FDI APIs using the same context will cause the ERF structure to be filled out.

Back to: Top of page > FDI > FDICreate

Returns

If successful, a non-NULL HFDI context pointer is returned. If unsuccessful, NULL is returned, and the error structure pointed to by *perf* is filled out.

Back to: Top of page > FDI > FDICreate

FDIIsCabinet

Back to: Top of page

```
BOOL DIAMONDAPI FDIIsCabinet(
      HFDI             hfdi,
      int              hf,
      PFDICABINETINFO  pfdici
);
```

Back to: Top of page > FDI > FDIIsCabinet

Parameters

hfdi

FDI Context pointer originally returned by **FDICreate**

hf

File handle returned by a call to the application's file open function

pfdici

Pointer to a cabinet info structure

Back to: Top of page > FDI > FDIIsCabinet

Description

The **FDIIsCabinet** API determines whether a given file is a cabinet, and if so, returns information about the cabinet in the provided FDICABINETINFO structure.

The *hfdi* parameter is the context pointer returned by a previous call to **FDICreate**.

The *hf* parameter must be a file handle on the file being examined. The file handle must be of the same type as those used by the file i/o functions passed to **FDICreate**.

The *pfdici* parameter should point to an FDICABINETINFO structure, which will receive the cabinet details if the file is indeed a cabinet. The fields of this structure are as follows:

The cbCabinet field contains the length of the cabinet file, in bytes. The cFolders field contains the number of folders in the cabinet. The cFiles field contains the total number of files in the cabinet. The setID field contains the set ID (an application-defined magic number) of the cabinet. The iCabinet field contains the number of this cabinet in the set (0 for the first cabinet, 1 for the second, and so forth). The fReserve field is a Boolean indicating whether there is a reserved area present in the cabinet. The hasprev field is a Boolean indicating whether this cabinet is chained to the previous cabinet, by way of having a file continued from the previous cabinet into the current one. The hasnext field is a Boolean indicating whether this cabinet is chained to the next cabinet, by way of having a file continued from this cabinet into the next one.

Back to: Top of page > FDI > FDIIsCabinet

Returns

If the file is a cabinet, then TRUE is returned and the FDICABINETINFO structure is filled out. If the file is not a cabinet, or some other error occurred, then FALSE is returned. In either case, it is the responsibility of the application to close the file handle passed to this function.

Back to: Top of page > FDI > FDIIsCabinet

FDICopy

Back to: Top of page

```
BOOL FAR DIAMONDAPI FDICopy(
        HFDI           hfdi,
    char  FAR         *pszCabinet,
    char  FAR         *pszCabPath,
    int                flags,
        PFNFDINOTIFY   pfnfdin,
        PFNFDIDECRYPT  pfnfdid,
    void  FAR         *pvUser
);
```

Back to: Top of page > FDI > FDICopy

Parameters

hfdi

FDI Context pointer originally returned by **FDICreate**

pszCabinet

Name of cabinet file, excluding path information

pszCabPath

File path to cabinet file

flags

Flags to control the extract operation

pfnfdin

Pointer to a notification (status update) function

pfnfdid

Pointer to a decryption function

pvUser

Application-specified value to pass to notification function

Back to: Top of page > FDI > FDICopy

Description

The **FDICopy** API extracts one or more files from a cabinet. Information on each file in the cabinet is passed back to the supplied pfnfdin function, at which point the application may decide to extract or not extract the file.

The *hfdi* parameter is the context pointer returned by a previous call to **FDICreate**.

The *pszCabinet* parameter should be the name of the cabinet file, excluding any path information, from which to extract files. If a file is split over multiple cabinets, **FDICopy** does allow subsequent cabinets to be opened.

The *pszCabPath* parameter should be the file path of the cabinet file (e.g. "C:\MYCABS\"). The contents of *pszCabPath* and *pszCabinet* will be

strung together to create the full pathname of the cabinet.

The *flags* parameter is used to set flags for the decoder. At this time there are no flags defined, and the *flags* parameter should be set to zero.

The *pfnfdin* parameter should point to a file notification function, which will be called periodically to update the application on the status of the decoder. The *pfnfdin* function takes two parameters; fdint, an integral value indicating the type of notification message, and pfdin, a pointer to an FDINOTIFICATION structure.

The fdint parameter may equal one of the following values; ***fdintCABINET_INFO*** (general information about the cabinet), ***fdintPARTIAL_FILE*** (the first file in the cabinet is a continuation from a previous cabinet), ***fdintCOPY_FILE*** (asks the application if this file should be copied), ***fdintCLOSE_FILE_INFO*** (close the file and set file attributes, date, etc.), or ***fdintNEXT_CABINET*** (file continued on next cabinet).

The pfdin parameter will point to an FDINOTIFICATION structure with some or all of the fields filled out, depending on the value of the fdint parameter. Four of the fields are used for general data; cb (a long integer), and psz1, psz2, and psz3 (pointers to strings), the meanings of which are highly dependent on the fdint value. The pv field will be the value the application originally passed in as the *pvUser* parameter to **FDICopy**.

The *pfnfdin* function must return a value to FDI, which tells FDI whether to continue, abort, skip a file, or perform some other operation. The values that can be returned depend on fdint, and are explained below.

Note that it is possible that future versions of FDI will have additional notification messages. Therefore, the application should ignore values of fdint it does not understand, and return zero to continue (preferably), or -1 (negative one) to abort.

If fdint equals ***fdintCABINET_INFO*** then the following fields will be filled out; psz1 will point to the name of the next cabinet (excluding path information); psz2 will point to the name of the next disk; psz3 will point to the cabinet path name; setID will equal the set ID of the current cabinet; and iCabinet will equal the cabinet number within the cabinet set (0 for the first cabinet, 1 for the second cabinet, etc.) The application should return 0 to indicate success, or -1 to indicate failure, which will abort **FDICopy**. An ***fdintCABINET_INFO*** notification will be provided exactly once for each cabinet opened by **FDICopy**, including continuation cabinets opened due to files spanning cabinet boundaries.

If fdint equals ***fdintCOPY_FILE*** then the following fields will be filled out; psz1 will point to the name of a file in the cabinet; cb will equal the uncompressed size of the file; date will equal the file's 16-bit FAT date; time will equal the file's 16-bit FAT time; and attribs will equal the file's 16-bit FAT attributes. The application may return one of three values; 0 (zero) to skip (i.e. not copy) the file; -1 (negative one) to abort **FDICopy**; or a non-zero (and non-negative-one) file handle for the destination to which to write the file. The file handle returned must be compatible with the PFNCLOSE function supplied to **FDICreate**. The ***fdintCOPY_FILE*** notification is called for each file that *starts* in the current cabinet, providing the opportunity for the application to request that the file be copied or skipped.

If fdint equals ***fdintCLOSE_FILE_INFO*** then the following fields will be filled out; psz1 will point to the name of a file in the cabinet; hf will be a file handle (which originated from ***fdintCOPY_FILE***); date will equal the file's 16-bit FAT date; time will equal the file's 16-bit FAT time; attributes will equal the file's 16-bit FAT attributes (minus the _A_EXEC bit); and cb will equal either zero (0) or one (1), indicating whether the file should be executed after extract (one), or not (zero). It is the responsibility of the application to execute the file if cb equals one. The ***fdintCLOSE_FILE_INFO*** notification is called after all of the data has been written to a target file. The application must close the file (using the provided hf handle), and set the file date, time, and attributes. The application should return TRUE for success, or FALSE or -1 (negative one) to abort **FDICopy**. FDI assumes that the target file was closed, even if this callback returns failure; FDI will not attempt to use PFNCLOSE to close the file.

If fdint equals ***fdintPARTIAL_FILE*** then the following fields will be filled out; psz1 will point to the name of the file continued from a previous cabinet; psz2 will point to the name of the cabinet on which the first segment of the file exists; psz3 will point to the name of the disk on which the first segment of the file exists. The ***fdintPARTIAL_FILE*** notification is called for files at the beginning of a cabinet which are continued from a previous cabinet. This notification will occur only when **FDICopy** is started on the second or subsequent cabinet in a series, which has files continued from a previous cabinet. The application should return zero (0) for success, or -1 (negative one) for failure, which will abort **FDICopy**.

If fdint equals ***fdintNEXT_CABINET*** then the following fields will be filled out; psz1 will point to the name of the next cabinet on which the current file is continued; psz2 will point to the name of the next disk on which the current file is continued; psz3 will point to the cabinet path information; and fdie will equal a success or error value. The ***fdintNEXT_CABINET*** notification is called only when ***fdintCOPY_FILE*** was instructed to copy a file in the current cabinet that is continued in a subsequent cabinet. It is important that the cabinet path name, psz3, be validated before returning (psz3, which points to a 256 byte array, may be modified by the application; however, it is not permissible to modify psz1 or psz2). The application should ensure that the cabinet exists and is readable before returning; if necessary, the application should issue a disk change prompt and ensure that the cabinet file exists. When this function returns to FDI, FDI will verify that the setID and iCabinet fields of the supplied cabinet match the expected values for that cabinet. If not, FDI will continue to send ***fdintNEXT_CABINET*** notification messages with the fdie field set to ***FDIERROR_WRONG_CABINET***, until the correct cabinet file is specified, or until this function returns -1 (negative one) to abort the **FDICopy** call. If after returning from this function, the cabinet file is not present and readable, or has been damaged, then the fdie field will equal one of the following values; ***FDIERROR_CABINET_NOT_FOUND***, ***FDIERROR_NOT_A_CABINET***, ***FDIERROR_UNKNOWN_CABINET_VERSION***, ***FDIERROR_CORRUPT_CABINET***, ***FDIERROR_BAD_COMPR_TYPE***, ***FDIERROR_RESERVE_MISMATCH***, ***FDIERROR_WRONG_CABINET***. If there was no error, fdie will equal ***FDIERROR_NONE***. The application should return 0 (zero) to indicate success, or -1 (negative one) to indicate failure, which will abort **FDICopy**.

The *pfndid* parameter is reserved for encryption, and is currently not used by FDI. This parameter should be set to NULL.

The *pvUser* parameter should contain an application-defined value that will be passed back as a field in the FDINOTIFICATION structure of the notification function. It not required, this field may be safely set to NULL.

Back to: Top of page > FDI > FDICopy

Returns

If successful, TRUE is returned. If unsuccessful, FALSE is returned, and the error structure pointed to by *perf* (from **FDICreate**) is filled out.

Back to: Top of page > FDI > FDICopy

FCIDestroy

Back to: Top of page

```
BOOL DIAMONDAPI FDIDestroy(
      HFDI  hfdi
);
```

Back to: Top of page > FDI > FDIDestroy

Parameters

hfdi

FDI Context pointer originally returned by **FDICreate**

Back to: Top of page > FDI > FDIDestroy

Description

The **FDIDestroy** API destroys an *hfdi* context, freeing any memory and temporary files associated with the context.

Back to: Top of page > FDI > FDIDestroy

Returns

If successful, TRUE is returned. If unsuccessful, FALSE is returned. The only reason for failure is that the *hfdi* passed in was not a proper context handle.

Back to: Top of page > FDI > FDIDestroy

#Microsoft LZX Data Compression Format

Copyright © 1997 Microsoft Corporation. All rights reserved.

Topics in this section

Introduction

Concepts

- LZ77
- Bitstream
- Window Size
- Trees
- Repeated Offsets
- Constants

LZX Compressed Data Format

- Cabinet Block Size
- Header Structure
- Encoder Preprocessing
- Block Structure
- Uncompressed Block Format
- Verbatim Block
- Aligned Offset Block
- Encoding the Trees and Pre-Trees
- Compressed Literals
- Match Offset => Formatted Offset
- Formatted Offset => Position Slot, Position Footer
- Position Footer => Verbatim Bits, Aligned Offset Bits
- Match Length => Length Header, Length Footer
- Length Header, Position Slot => Length/Position Header
- Encoding a Match

Decoding a Match or an Uncompressed Character

Introduction

This document is a design specification for the format of LZX compressed data used in the LZX compression mode of Microsoft's CAB file format. The purpose of this document is to allow anyone to encode or decode LZX compressed data. This document describes only the format of the output –it does not provide any specific algorithms for match location, tree generation, etc.

Before proceeding with the design specification itself, a few important concepts are described in the following pages.

Back to: Top of page

Concepts

This section includes:

LZ77

Bitstream

Window Size

Trees

Repeated Offsets

Constants

Back to: Top of page

LZ77

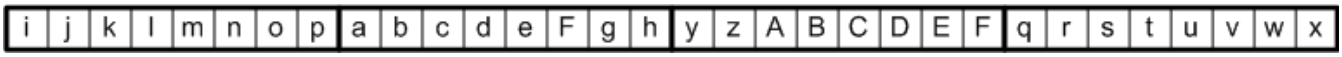
LZX is an LZ77 based compressor that uses static Huffman encoding and a sliding window of selectable size. Data symbols are encoded either as an uncompressed symbol, or as an (offset, length) pair indicating that *length* symbols should be copied from a displacement of -*offset* symbols from the current position in the output stream. The value of *offset* is constrained to be less than the size of the sliding window.

Back to: Top of page > Concepts

Bitstream

An LZX bitstream is a sequence of 16 bit integers stored in the order least-significant-byte most-significant-byte. Given an input stream of bits named a, b, c, ..., x, y, z, A, B, C, D, E, F, the output byte stream (with byte boundaries highlighted) would be as shown below.

Output byte stream



Back to: Top of page > Concepts

Window Size

The window size must be a power of 2, from 2^{15} to 2^{21}. The window size is not stored in the compressed data stream, and must instead be passed to the decoder before decoding begins.

The window size determines the number of window subdivisions, or "position slots", as shown in the following table:

Windows Size / Position Slot Table

Window Size	Position Slots Required
32K	30
64K	32
128K	34
256K	36
512K	38
1 MB	40
2 MB	42

Back to: Top of page > Concepts Trees

LZX uses canonical Huffman tree structures to represent elements. Huffman trees are well known in data compression and are not described here. Since an LZX decoder uses only the path lengths of the Huffman tree to reconstruct the identical tree, the following constraints are made on the tree structure:

1. For any two elements with the same path length, the lower-numbered element must be further left on the tree than the higher numbered element. An alternative way of stating this constraint is that lower-numbered elements must have lower path traversal values; for example, 0010 (left-left-right-left) is lower than 0011 (left-left-right-right).
2. For each level, starting at the deepest level of the tree and then moving upwards, leaf nodes must start as far left as possible. An alternative way of stating this constraint is that if any tree node has children then all tree nodes to the left of it with the same path length must also have children.
3. Zero length Huffman codes are not permitted, therefore a tree must contain at least 2 elements. In the case where all tree elements are zero frequency, or all but one tree element is zero frequency, the resulting tree must consist of the two Huffman codes "0" and "1". In the latter case, constraint #1 still applies.

LZX uses several Huffman tree structures. The most important tree is the *main* tree, which comprises 256 elements corresponding to all

possible ASCII characters, plus 8 * NUM_POSITION_SLOTS (see above) elements corresponding to matches. The second most important tree is the *length* tree, which comprises 249 elements.

Other trees, such as the aligned offset tree (comprising 8 elements), and the pre-trees (comprising 20 elements each), have a smaller role.

Back to: Top of page > Concepts

Repeated Offsets

LZX extends the conventional LZ77 format in several ways, one of which is in the use of repeated offset codes. Three match offset codes, named the *repeated offset codes*, are reserved to indicate that the current match offset is the same as that of one of the three previous matches which is not itself a repeated offset.

The three special offset codes are encoded as offset values 0, 1, and 2 (i.e. encoding an offset of 0 means "use the most recent non-repeated match offset", an offset of 1 means "use the second most recent non-repeated match offset", etc.). All remaining offset values are displaced by +3, as is shown in the table below, which prevents matches at offsets WINDOW_SIZE, WINDOW_SIZE-1, and WINDOW_SIZE-2.

Correlation Between Encoded Offset and Real Offset

Encoded Offset	Real Offset
0	Most recent non-repeated match offset
1	Second most recent non-repeated match offset
2	Third most recent non-repeated match offset
3	1 (closest allowable)
4	2
5	3
6	4
7	5
8	6
500	498
x+2	x
WINDOW_SIZE-1	WIDOW_SIZE-3

(maximum possible)

The three most recent non-repeated match offsets are kept in a list, the behavior of which explained below:

> Let R0 be defined as the most recent non-repeated offset
>
> Let R1 be defined as the second most recent non-repeated offset
>
> Let R2 be defined as the third most recent non-repeated offset

The list is managed similarly to an LRU (least recently used) queue, with the exception of the cases when R1 or R2 is output. In these cases, which are fairly uncommon, R1 or R2 is simply swapped with R0, which requires fewer operations than would an LRU queue. The compression penalty from doing so is essentially zero and it removes a small computational overhead from the decoder.

The initial state of R0, R1, R2 is (1, 1, 1).

Management of the Repeated Offsets List

Match Offset *X* where...	Operation
X ≠ R0 and X ≠ R1 and X ≠ R2	R2 ← R1 R1 ← R0 R0 ← X
X = R0	None
X = R1	Swap R0 ⇔ R1
X = R2	Swap R0 ⇔ R2

Back to: Top of page > Concepts

Constants

The following named constants are used frequently in this document:

Constant	Description	Value
MIN_MATCH	Smallest allowable match length	2
MAX_MATCH	Largest allowable match length	257
NUM_CHARS	Number of uncompressed character types	256
WINDOW_SIZE	Window size	Varies
NUM_POSITION_SLOTS	Number of window subdivisions	Dependent upon WINDOW_SIZE
MAIN_TREE_ELEMENTS	Number of elements in main tree	NUM_CHARS + NUM_POSITION_SLOTS*8
NUM_SECONDARY_LENGTHS	Number of elements in length tree	249

Back to: Top of page > Concepts

LZX Compressed Data Format

LZX compressed data consists of a header indicating the file translation size (which is described later), followed by a sequence of compressed blocks. A stream of uncompressed input may be output as multiple compressed LZX blocks to improve compression, since each compressed block contains its own statistical tree structures.



This section includes:

- Cabinet Block Size
- Header Structure
- Encoder Preprocessing
- Block Structure
- Uncompressed Block Format
- Verbatim Block
- Aligned Offset Block
- Encoding the Trees and Pre-Trees
- Compressed Literals
- Match Offset ⇒ Formatted Offset
- Formatted Offset ⇒ Position Slot, Position Footer
- Position Footer ⇒ Verbatim Bits, Aligned Offset Bits
- Match Length ⇒ Length Header, Length Footer
- Length Header, Position Slot ⇒ Length/Position Header
- Encoding a Match
- Decoding a Match or an Uncompressed Character

Back to: Top of page

Cab Block Size

The cabinet file format requires that for any particular CFDATA block, the indicated number of compressed input bytes must represent exactly the indicated number of uncompressed output bytes. Furthermore, each CFDATA block must represent 32768 uncompressed bytes, with the exception of the last CFDATA block in a folder, which may represent less than 32768 uncompressed bytes.

The LZX block size is independent of the CFDATA block size; an LZX block can represent 200,000 uncompressed bytes, for example. In order to ensure that an exact number of input bytes represent an exact number of output bytes, after each 32768th uncompressed byte is represented, the output bit buffer is byte aligned on a 16-bit boundary by outputting 0-15 zero bits. The bit buffer is flushed in an identical manner after the final CFDATA block in a folder. Furthermore, the compressor may not emit any matches that span a 32768-byte boundary in the input (for example, at position 65528 in the input, the compressor cannot emit a match with a length of 50; the maximum allowable match length at this point would be 6).

One additional constraint is that, for any given CFDATA block, the compressed size of a CFDATA block may not occupy more than 32768+6144 bytes (i.e. 32K of uncompressed input may not grow by more than 6K when compressed).

Back to: Top of page > LZX Compressed Data Format

Header Structure

The header consists of either a zero bit indicating no encoder preprocessing, or a one bit followed by a *file translation size*, a value which is used in encoder preprocessing.

0
1 Most significant 16 bits of file translation size Least significant 16 bits of file translation size

Back to: Top of page > LZX Compressed Data Format

Encoder preprocessing

The encoder may optionally perform a preprocessing stage on all CFDATA input blocks (size <= 32K) which improves compression on 32-bit Intel 80x86 code. The translation is performed before the data is passed to the compressor, and therefore an appropriate reverse translation must be performed on the output of the decompressor. A bit indicating whether preprocessing was used is stored in the compression header (see above).

The preprocessing stage translates 80x86 CALL instructions, which begin with the E8 (hex) opcode, to use absolute offsets instead of relative offsets.

Preprocessing is disabled after the 32768th CAB input frame in a folder (where a CAB input frame is 32768 bytes) in order to avoid signed/unsigned arithmetic complexity. This change can obviously occur only when a folder represents at least 1 gigabyte of uncompressed data.

CALL Byte Sequence (E8 followed by 32 bit offset)

```
E8 r0 r1 r2 r3
```

Performing the Relative-to-Absolute Conversion

```
relative_offset ← r0 + r1*2^8 + r2*2^16 + r3*2^24
new_value ← conversion_function(current_location, relative_offset)
a0 ← bits 0-7 of new_value
a1 ← bits 8-15 of new_value
a2 ← bits 16-23 of new_value
a3 ← bits 24-31 of new_value
```

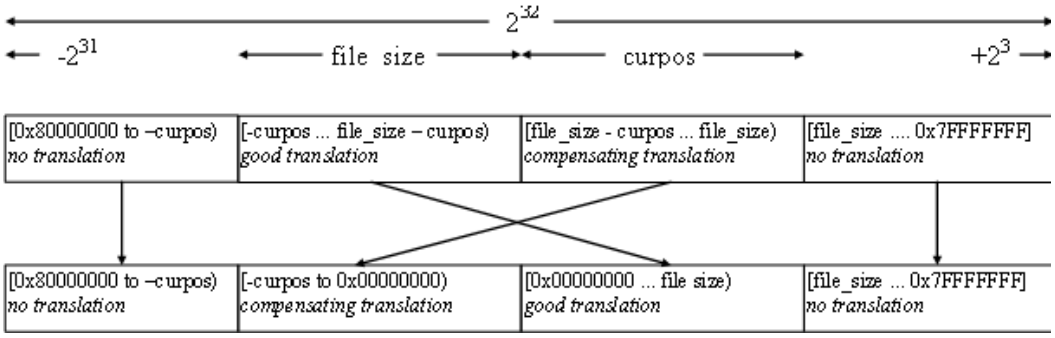
Translated CALL Byte Sequence

```
E8 a0 a1 a2 a3
```

The diagram below illustrates the relative-to-absolute conversion function, where *curpos* is the current offset within all uncompressed data seen in the current cabinet folder, and *file_size* is the file translation size from the compression header (*file_size* is unrelated to the size of the actual file being decompressed).

The translation is performed "in place" on the input data without using extra codes to indicate whether a translation occurred (i.e. there is a direct mapping from a 32-bit value to a 32-bit value), therefore there is a one-to-one correlation between pre- and post- translated values.

Offset Translation Diagram



From the diagram one can see that values in the range of *0x80000000 (-2^{31})* to *-curpos*, and *file_size* to *0x7FFFFFFF (+2^{31})* are left unchanged. The translation algorithm operates as follows on an input block of size *input_size*, where 0 <= *input_size* <= 32768. No translation may be performed on the last 6 bytes of the input block.

```
if (input_size < 6)
return          /* don't perform translation if < 6 input bytes */

for (i = 0; i < input_size; i++)

   if (input_data[i] == 0xE8)
      if (i >= input_size-6)
   break;
      endif

      ... perform translation illustrated above Â…
   endif
```

Back to: Top of page > LZX Compressed Data Format

Block Structure

Each block of compressed data begins with a 3 bit header describing the block type, followed by the block itself. The allowable block types are:

0	Undefined
1	Verbatim block
2	Aligned offset block
3	Uncompressed block
4-7	Undefined

Back to: Top of page > LZX Compressed Data Format

Uncompressed Block Format

An uncompressed block begins with 1 to 16 bits of zero padding to align the bit buffer on a 16-bit boundary. At this point, the bitstream ends, and a *bytestream* begins. The data that follows is encoded as bytes for performance. Following the zero padding, new values for R_0, R_1, and R_2 are output in little-endian form, followed by the uncompressed data bytes themselves.

1-16 bits	4 bytes	4 bytes	4 bytes	*n* bytes
zero padding	R0	R1	R2	Uncompressed data
	(LSB first)	(LSB first)	(LSB first)	

Back to: Top of page > LZX Compressed Data Format

Verbatim Block

A verbatim block consists of the following:

Entry	Comments	Size
Number of uncompressed bytes accounted for in this block	Range of $1...2^{24}$	24 bits
Pre-tree for first 256 elements of main tree	20 elements, 4 bits each	80 bits
Path lengths of first 256 elements of main tree	Encoded using pre-tree	Variable
Pre-tree for remainder of main tree	20 elements, 4 bits each	80 bits
Path lengths of remaining elements of main tree	Encoded using pre-tree	Variable
Pre-tree for length tree	20 elements, 4 bits each	80 bits
Path lengths of elements in length tree	Encoded using pre-tree	Variable
Compressed literals	Described later	Variable

Back to: Top of page > LZX Compressed Data Format

Aligned Offset Block

An aligned offset block consists of the following:

Entry	Comments	Size
Number of uncompressed bytes accounted for in this block	Range of $1...2^{24}$	24 bits
Pre-tree for first 256 elements of main tree	20 elements, 4 bits each	80 bits
Path lengths of first 256 elements of main tree	Encoded using pre-tree	Variable
Pre-tree for remainder of main tree	20 elements, 4 bits each	80 bits
Path lengths of remaining elements of main tree	Encoded using pre-tree	Variable
Aligned offset tree	8 elements, 3 bits each	24 bits
Compressed literals	Described later	Variable

The aligned offset tree comprises only 8 elements, each of which is encoded as a 3 bit path length. Since the size of this tree is so small, no additional compression is performed on it.

Back to: Top of page > LZX Compressed Data Format

Encoding the Trees and Pre-Trees

Since all trees used in LZX are created in the form of a canonical Huffman tree, the path length of each element in the tree is sufficient to reconstruct the original tree. The main tree and the length tree are each encoded using the method described below. However, the main tree is encoded in two components as if it were two separate trees, the first tree corresponding to the first 256 tree elements (uncompressed symbols), and the second tree corresponding to the remaining elements (matches).

Since trees are output several times during compression of large amounts of data, LZX optimizes compression by encoding only the delta path lengths between the current and previous trees. In the case of the very first such tree, the delta is calculated against a tree in which all elements have a zero path length.

Each tree element may have a path length from 0 to 16 (inclusive) where a zero path length indicates that the element has a zero frequency and is not present in the tree. Tree elements are output in sequential order starting with the first element. Elements may be encoded in one of two ways -if several consecutive elements have the same path length, then run length encoding is employed; otherwise the element is output by encoding the difference between the current path length and the previous path length of the tree, mod 17. These output methods are described below:

Tree Codes

Code	Operation
0-16	Len[x] = (prev_len[x] + code) mod 17
17	Zeroes = getbits(4) Len[x] = 0 for next (4 + Zeroes) elements
18	Zeroes = getbits(5) Len[x] = 0 for next (20 + Zeroes) elements
19	Same = getbits(1) Decode new Code Value = (prev_len[x] + Code) mod 17 Len[x] = Value for next (4 + Same) elements

Each of the 17 possible values of (*len[x] - prev_len[x]) mod 17*, plus three additional codes used for run-length encoding, are *not* output directly as 5 bit numbers, but are instead encoded via a Huffman tree called the *pre- tree*. The pre-tree is generated dynamically according to the frequencies of the 20 allowable tree codes. The structure of the pre-tree is encoded in a total of 80 bits by using 4 bits to output the path length of each of the 20 pre-tree elements. Once again, a zero path length indicates a zero frequency element.

Pre-Tree

Length of tree code 0	4 bits
Length of tree code 1	4 bits
Length of tree code 2	4 bits

... ...
Length of tree code 18 4 bits
Length of tree code 19 4 bits

The "real" tree is then encoded using the pre-tree Huffman codes.

Back to: Top of page > LZX Compressed Data Format

Compressed Literals

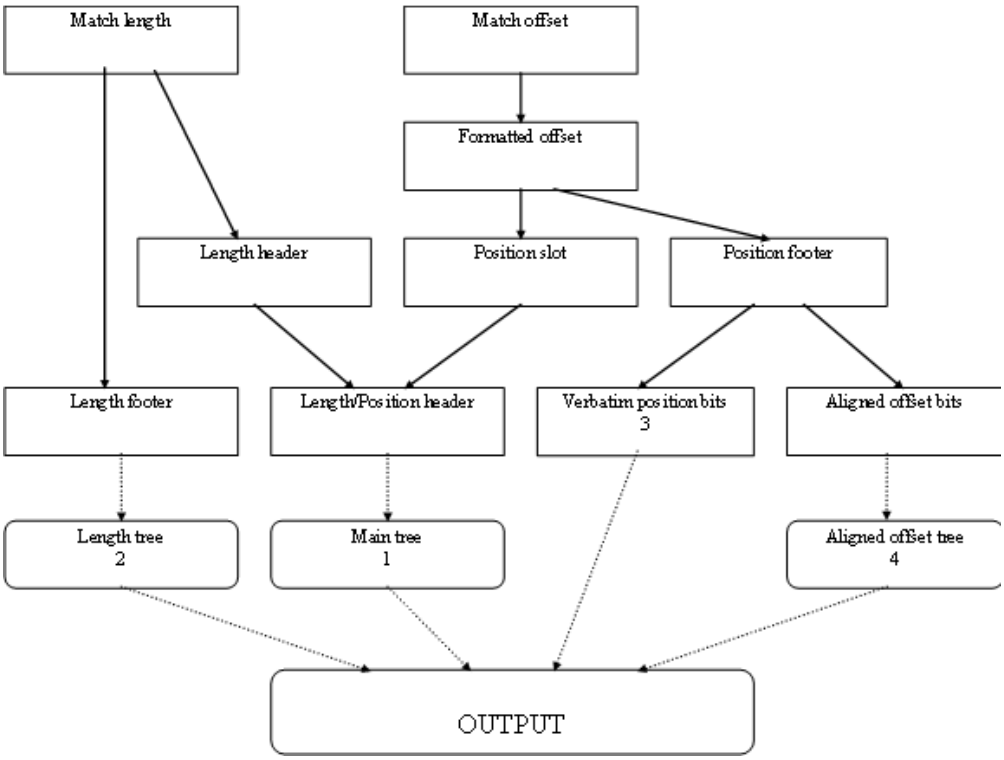
The compressed literals that make up the bulk of either a verbatim block or an aligned offset block immediately follow the tree data (as shown in the diagram for each block type). These literals, which comprise matches and unmatched characters, will, when decompressed, correspond to exactly the number of uncompressed bytes indicated in the block header.

The representation of an unmatched character in the output is simply the appropriate element 0Â…(NUM_CHARS-1) Huffman-encoded using the main tree.

The representation of a match in the output involves several transformations, as shown in the following diagram. At the top of the diagram are the match length (MIN_MATCHÂ…MAX_MATCH) and the match offset (0Â…WINDOW_SIZE-4). The match offset and match length are split into sub-components and encoded separately.

As mentioned previously, in order to remain compatible with the cabinet file format, the compressor may not emit any matches that span a 32768-byte boundary in the input.

Diagram of Match Sub-Components



Back to: Top of page > LZX Compressed Data Format

Match Offset ⇒ Formatted Offset

The match offset, range 1...(WINDOW_SIZE-4), is converted into a *formatted offset* by determining whether the offset can be encoded as a repeated offset, as shown below. It is acceptable to not encode a match as a repeated offset even if it is possible to do so.

Converting a Match Offset to a Formatted Offset

```
if offset == R0 then
   formatted offset ← 0
else if offset == R1 then
   formatted offset ← 1
else if offset == R2 then
   formatted offset ← 2
else
   formatted offset ← offset + 2
endif
```

Back to: Top of page > LZX Compressed Data Format

Formatted Offset ⇒ Position Slot, Position Footer

The formatted offset is subdivided into a position slot and a position footer. The position slot defines the most significant bits of the formatted offset in the form of a base position as shown in the table on the following page. The position footer defines the remaining least significant bits of the formatted offset. As the table shows, the number of bits dedicated to the position footer grows as the formatted offset becomes larger, meaning that each position slot addresses a larger and larger range.

The number of position slots available depends on the window size. The position slot table for the maximum window size of 2 megabytes, is shown in the table below.

Position Slot Table

Position Slot Number	Base Position	Number of Position Footer Bits	Range of Base Position and Position Footer
0	0	0	0
1	1	0	1
2	2	0	2
3	3	0	3
4	4	1	4-5
5	6	1	6-7
6	8	2	8-11
7	12	2	12-15
8	16	3	16-23
9	24	3	24-31
10	32	4	32-47
11	48	4	48-63
12	64	5	64-95
13	96	5	96-127
14	128	6	128-191
15	192	6	192-255
16	256	7	256-383
17	384	7	384-511
18	512	8	512-767
19	768	8	768-1023
20	1024	9	1024-1535
21	1536	9	1536-2047
22	2048	10	2048-3071
23	3072	10	3072-4095
24	4096	11	4096-6143
25	6144	11	6144-8191
26	8192	12	8192-12287
27	12288	12	12288-16383
28	16384	13	16384-24575
29	24576	13	24576-32767
30	32768	14	32768-49151
31	49152	14	49152-65535
32	65536	15	65536-98303
33	98304	15	98304-131071
34	131072	16	131072-196607
35	196608	16	196608-262143
36	262144	17	262144-393215
37	393216	17	393216-524287
38	524288	17	524288-655359
39	655360	17	655360-786431
40	786432	17	786432-917503
41	917504	17	917504-1048575
42	1048576	17	1048576-1179647
43	1179648	17	1179648-1310719
44	1310720	17	1310720-1441791
45	1441792	17	1441792-1572863
46	1572864	17	1572864-1703935
47	1703936	17	1703936-1835007
48	1835008	17	1835008-1966079
49	1966080	17	1966080-2097151

In order to determine the position footer, it is first necessary to determine the position slot. Then, a simple lookup can be performed on the position slot to determine the number of bits, B, in the position footer. The B least significant bits of the formatted offset are the position footer. Pseudocode for obtaining the position slot and position footer are shown below, as is the lookup array (named *extra_bits*).

n (position slot)	extra_bits[n] (number of position footer bits)
0	0
1	0
2	0
3	0
4	1
5	1
6	2
7	2
8	3
9	3

10	4
11	4
12	5
13	5
14	6
15	6
16	7
17	7
18	8
19	8
20	9
21	9
22	10
23	10
24	11
25	11
26	12
27	12
28	13
29	13
30	14
31	14
32	15
33	15
34	16
35	16
36-49	17

Converting the Position Slot and Position Footer

```
position_slot ← calculate_position_slot(formatted_offset)
position_footer_bits ← extra_bits[ position_slot ]
if position_footer_bits > 0
      position_footer ← formatted_offset & ((2^position_footer_bits)-1)
else
      position_footer ← null
```

Back to: Top of page > LZX Compressed Data Format

Position Footer ⇒ Verbatim Bits, Aligned Offset Bits

The position footer may be further subdivided into verbatim bits and aligned offset bits if the current block uses aligned offsets. If the current block is not an aligned offset block then there are no aligned offset bits, and the verbatim bits are the position footer.

If aligned offsets are used, then the lower 3 bits of the position footer are the aligned offset bits, while the remaining portion of the position footer are the verbatim bits. In the case where there are less than 3 bits in the position footer (i.e. formatted offset is <= 15) it is not possible to take the "lower 3 bits of the position footer" and therefore there are no aligned offset bits, and the verbatim bits and the position footer are the same.

Pseudocode for Splitting Position Footer into Verbatim Bits and Aligned Offset

```
if block_type = aligned_offset_block then
   if formatted_offset <= 15 then
      verbatim_bits ← position_footer
      aligned_offset ← null
   else
      aligned_offset ← position_footer
      verbatim_bits ← position_footer >> 3
   endif
else
   verbatim_bits ← position_footer
   aligned_offset ← null
endif
```

Back to: Top of page > LZX Compressed Data Format

Match Length ⇒ Length Header, Length Footer

The match length is converted into a length header and a length footer. The length header may have one of eight possible values, from 0...7 (inclusive), indicating a match of length 2, 3, 4, 5, 6, 7, 8, or a length greater than 8. If the match length is 8 or less, then there is no length footer. Otherwise the value of the length footer is equal to the match length minus 9.

Pseudocode for Obtaining the Length Header and Footer

```
if match_length <= 8
   length_header ← match_length-2
   length_footer ← null
else
   length_header ← 7
   length_footer ← match_length-9
endif
```

Example Conversions of Some Match Lengths to Header and Footer Values

Match length	Length header	Length footer value

2 (MIN_MATCH)	0	None
3	1	None
4	2	None
5	3	None
6	4	None
7	5	None
8	6	None
9	7	0
10	7	1
50	7	41
257 (MAX_MATCH)	7	248

Back to: Top of page > LZX Compressed Data Format

Length Header, Position Slot ⇒ Length / Position Header

The Length/Position header is the stage which correlates the match position with the match length (using only the most significant bits), and is created by combining the length header and the position slot as shown below:

```
len_pos_header ← (position_slot < < 3) + length_header
```

This operation creates a unique value for every combination of match length 2, 3, 4, 5, 6, 7, 8 with every possible position slot. The remaining match lengths greater than 8 are all lumped together, and as a group are correlated with every possible position slot.

Back to: Top of page > LZX Compressed Data Format

Encoding a Match

The match is finally output in up to four components, as follows:

1. Output element (len_pos_header + NUM_CHARS) from the main tree
2. If length_footer != null, then output element length_footer from the length tree
3. If verbatim_bits != null, then output verbatim_bits
4. If aligned_offset_bits != null, then output element aligned_offset from the aligned offset tree

Back to: Top of page > LZX Compressed Data Format

Decoding a Match or an Uncompressed Character

Decoding is performed by first decoding an element using the main tree and then, if the item is a match, determining which additional components are necessary to reconstruct the match. Pseudocode for decoding a match or an uncompressed character is shown below:

```
main_element = main_tree.decode_element()

if (main_element < NUM_CHARS) /* is an uncompressed character */

   window[ curpos ] ← (byte) main_element
   curpos ← curpos + 1

else /* is a match */

      length_header ← (main_element - NUM_CHARS) & NUM_PRIMARY_LENGTHS

      if (length_header == NUM_PRIMARY_LENGTHS)
            match_length ← length_tree.decode_element() + NUM_PRIMARY_LENGTHS + MIN_MATCH
      else
            match_length ← length_header + MIN_MATCH /* no length footer */
      endif

      position_slot ← (main_element - NUM_CHARS) >> 3

      /* check for repeated offsets (positions 0,1,2) */
      if (position_slot == 0)
            match_offset ← R0
      else if (position_slot == 1)
            match_offset ← R1
            swap(R0 ⇔ R1)
      else if (position_slot == 2)
            match_offset ← R2
            swap(R0 ⇔ R2)
      else /* not a repeated offset */
            extra ← extra_bits[ position_slot ]

            if (block_type == aligned_offset_block)
                  if (extra > 3) /* this means there are some aligned bits */
                        verbatim_bits ← (readbits(extra-3)) << 3
                        aligned_bits  ← aligned_offset_tree.decode_element();
                  else if (extra > 0) /* just some verbatim bits */
                        verbatim_bits ← readbits(extra)
                        aligned_bits  ← 0
                  else /* no verbatim bits */
                        verbatim_bits ← 0
                        aligned_bits  ← 0
            endif

            formatted_offset ← base_position[ position_slot ] + verbatim_bits + aligned_bits
      else /* block_type == verbatim_block */
            if (extra > 0) /* if there are any extra bits */
                  verbatim_bits ← readbits(extra)
            else
                  verbatim_bits ← 0
            endif

            formatted_offset ← base_position[ position_slot ] + verbatim_bits
      endif

      match_offset ← formatted_offset - 2
```

```
        /* update repeated offset LRU queue */
        R2 ← R1
        R1 ← R0
        R0 ← match_offset

        /* copy match data */
        for (i = 0; i < match_length; i++)
              window[curpos + i] ← window[curpos + i - match_offset]

        curpos ? curpos + match_length
 endif
```


Microsoft MakeCAB User's Guide

Copyright © 1997 Microsoft Corporation. All rights reserved.

Topics in this Section

Overview

- Case 1: MakeCAB for Setup Programs
  - Characteristics of a Setup Program
  - MakeCAB Application
- Case 2: MakeCAB for a 200MB Source Code Archive
  - Characteristics of a Source Code Archive
  - MakeCAB Application
- Case 3: Self-extracting Cabinet File(s)
- MakeCAB Deliverables
- MakeCAB Goals

MakeCAB Optimizing and Tuning

- Saving Diskettes
- Tuning Access Time vs. Compression Ratio
- Piecemeal DDFs for Localization and Different Disk Sizes

MakeCAB Concepts

- Decoupling File Layout and INF Layout

MAKECAB.EXE

- MAKECAB.EXE Syntax
- MAKECAB.EXE Directive File Syntax
- Command Summary
- Variable Summary
- InfDisk/Cabinet/FileLineFormat Syntax and Semantics
- INF Parameters
- Command Details
- Variable Details

EXTRACT.EXE

Overview

MakeCAB is a lossless data compression tool that can be used for a wide variety of purposes. Although it was originally designed for use by setup programs, it can also be used in almost any situation where lossless data compression is required.

MakeCAB has three key features: 1) storing multiple files in a single *cabinet ("CAB")* file, 2) performing compression **across** file boundaries, and 3) permitting files to span cabinets. While existing products such as PKZIP, LHARC, and ARJ, support some of these features, combining all three does not appear to be common practice. MakeCAB also supports self-extracting archives, by simply concatenating a cabinet file to EXTRACT.EXE.

Depending upon the number of files to be compressed, and the access patterns expected (sequential or random access; whether most of the files will be requested at once or only a small portion of them), MakeCAB can be instructed to build cabinet files in different ways. One key concept in MakeCAB is the *folder*. A *folder* is a collection of one or more files which are compressed together, as a single entity.

The cabinet file format is capable of supporting multiple forms of compression. At this time, MSZIP and LZX are the compression formats supported by Microsoft. Other compression formats are possible in the future.

The following sections provide case studies of several *possible* ways that MakeCAB might be used. These are only provided to stimulate your imagination -- they are not the only ways in which MakeCAB can be used!

Back to: Top of page

Case 1: MakeCAB for Setup Programs

Since MakeCAB was designed with setup programs in mind, it has a great deal of power and flexibility to tradeoff compressed size against speed of random access to files. The primary impact of MakeCAB is to minimize the number of diskettes required to distribute a product, thereby minimizing the Cost of Goods Sold (COGS).

In order for MakeCAB to build the disk images for a product, a *directive file*, or DDF, which specifies the list of files in a product, and any constraints on which disks certain files should be located, must be created. The same directive file can even be used for all the various localized versions of a product, since directive files support parameterization.

This section includes:

Characteristics of a Setup Program

MakeCAB Application

Back to: Top of page > Overview

Characteristics of a Setup Program

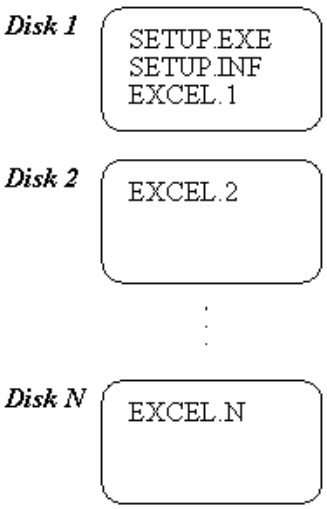
1. Minimizing disk count is very important, since it saves money in production costs.
2. Files are accessed sequentially.
3. Most files are accessed.

Back to: Top of page > Overview > Case 1: MakeCAB for Setup Programs

MakeCAB Application

The distribution disks for a typical application product produced by MakeCAB might look similar to the following:

Distribution disk layout



SETUP.EXE is the setup program, and SETUP.INF is a file generated by MakeCAB which guides the operation of the setup program (which files are needed for which options, and on which disk and in which cabinet file a file is contained). All of the remaining product files are contained in the cabinet files EXCEL.1 through EXCEL.N (N might be 7, for example).

To produce this disk layout with MakeCAB, a DDF is prepared which lists all of the files for the product, along with some optional MakeCAB settings to control parameters such as: 1) the capacity of the disks which are being used, 2) the naming convention of the cabinet files , 3) the visible (user-readable) labels on each disk, 4) how much random access is desired for files within a cabinet. The following is an example of a DDF that might be appropriate:

```
;*** MakeCAB Directive file example
;
.OPTION EXPLICIT                    ; Generate errors on variable typos

.Set DiskLabel1=Setup               ; Label of first disk
.Set DiskLabel2=Program             ; Label of second disk
.Set DiskLabel3="Program Continued" ; Label of third disk
.Set CabinetNameTemplate=EXCEL.*    ; EXCEL.1, EXCEL.2, etc.
.set DiskDirectoryTemplate=Disk*    ; disk1, disk2, etc.
.Set MaxDiskSize=1.44M              ; 3.5" disks

;** Setup.exe and setup.inf are placed uncompressed in the first disk
.Set Cabinet=off
.Set Compress=off
.Set InfAttr=                       ; Turn off read-only, etc. attrs
bin\setup.exe                       ; Just copy SETUP.EXE as is
bin\setup.inf                       ; Just copy SETUP.INF as is

;** The rest of the files are stored, compressed, in cabinet files
.Set Cabinet=on
.Set Compress=on
bin\excel.exe                       ; Big EXE, will span cabinets
bin\excel.hlp
bin\olecli.dll
bin\olesrv.dll
```

```
;...                                ; Many more files
;*** <the end>                      ; That's it
```

Now, you run MakeCAB to create the disk layout:

```
MakeCAB /f excel.ddf
```

MakeCAB will create directories Disk1, Disk2, etc. to hold the files for each disk, and will copy uncompressed files or create cabinet files (as appropriate) in each directory. The file SETUP.RPT will be written to the current directory (this can be overridden) with a summary of what MakeCAB did, and the file SETUP.INF will contain details on every disk and cabinet created, including a list of where each file was placed.

Back to: Top of page > Overview > Case 1: MakeCAB for Setup Programs

Case 2: MakeCAB for a 200MB Source Code Archive

The Microsoft Developers Network (MSDN) CD includes over 200Mb of source code. While uncompressed this is only 1/3rd of the CD, that is still too much space, so tight compression is desired. This is slightly different from the Setup case, however, since there is a front-end tool that allows users to select sample programs and expand them onto the hard disk.

This section includes:

Characteristics of a Source Code Archive

MakeCAB Application

Back to: Top of page

Characteristics of a Setup Program

1. Minimizing space usage is slightly less important
2. Files are accessed somewhat randomly, though in groups
3. Only a small portion of the files will be accessed at any one time

Back to: Top of page > Overview > Case 2: MakeCAB for a 200Mb Source Code Archive

MakeCAB Application

The cabinet files produced for the source archive need to be big enough to provide good compression, but not so big that random access speed is sacrificed. The challenge is to obtain a good tradeoff between compression and access time.

```
;*** MSDN Sample Source Code MakeCAB Directive file example
;
.OPTION EXPLICIT                   ; Generate errors on variable typos

.Set CabinetNameTemplate=MSDN.*    ; MSDN.1, MSDN.2, etc.
.set DiskDirectoryTemplate=CDROM   ; All cabinets go in a single directory
.Set MaxDiskFileCount=1000         ; Limit file count per cabinet, so that
                                   ; scanning is not too slow
.Set FolderSizeThreshold=200000    ; Aim for ~200K per folder
.Set CompressionType=MSZIP

;** All files are compressed in cabinet files
.Set Cabinet=on
.Set Compress=on
foo.c
foo.h
....
;*** <the end>                     ; That's it
```

Back to: Top of page > Overview > Case 2: MakeCAB for a 200Mb Source Code Archive

Case 3: Self-extracting Cabinet File(s)

Many times, a software developer will want to ship executables, libraries, or the like across an Intranet or the Internet. They need a small package and an easy way for users to extract data. For example, Java[TM] developers may want to ship large libraries of classes, so that home and business developers can use those classes in their software.

EXTRACT.EXE, which extracts files from CAB files, recognizes when it has been copied to the front of a cabinet file, and will automatically extract the files in that cabinet file (and any continuation cabinet files). Here is how this is accomplished:

1. Create a cabinet file (or set of cabinet files).
2. Prepend EXTRACT.EXE to the first cabinet file (do not prepend EXTRACT.EXE to any other cabinet files in the set).
3. Distribute the self-extracting cabinet (and any subsequent cabinets).

Example:

```
MakeCAB /f self.ddf                    ; Build cabinet file set self1.cab, self2.cab
copy /b extract.exe+self1.cab self.exe  ; self.exe is self-extracting
```

Back to: Top of page > Overview

MakeCAB Deliverables

The following table is a list of all the libraries and programs that are part of MakeCAB:

File	Contents
MAKECAB.EXE	Command-line tool to perform disk layout (uses FCI.LIB)

FDI.LIB	File Decompression Interface library.
EXTRACT.EXE	Command-line tool to expand files (uses FDI.LIB)
FCI.LIB	File Compression Interface library.

Back to: Top of page > Overview

MakeCAB Goals

- Provide excellent compression ratio and decompression speed
- Simplify production of disk layouts for products
- Provide command-line tools and link libraries for all Microsoft platforms

Back to: Top of page > Overview

MakeCAB Optimizing and Tuning

This section includes:

Saving Diskettes

Tuning Access Time vs. Compression Ratio

Piecemeal DDFs for Localization and Different Disk Sizes

Back to: Top of page

Saving Diskettes

For a product shipped on floppy disks, it is very important to minimize the number of disks shipped per product! As a back-of-the-envelope calculation, if each disk cost a dollar and one million units were shipped, then each disk saved would save $1 million. The following pseudo-code suggests a process you might follow as you strive to keep your Cost of Goods Sold (COGS) to a minimum:

```
get initial product files;
while (have not yet shipped)
   Compress file set using:
      CompressionType=LZX
      CompressionMemory=21
   If near a disk boundary
      Consider tossing files to save a disk (especially clipart & samples!)
   If near shipping
      Relax FolderSizeThreshold to
      improve access time at decompress.
end-while
Ship it!
```

Back to: Top of page > MakeCAB Optimizing and Tuning

Tuning Access Time vs. Compression Ratio

MakeCAB introduces the concept of a *folder* to refer to a contiguous set of compressed bytes. To decompress a file from a cabinet, FDI.LIB (called by your SETUP.EXE and EXTRACT.EXE) finds the folder that the file starts in, and then must **read and decompress** all the bytes in that folder from the start up through and including the desired file.

For example, if the file FOO.EXE is at the end of a 1.44Mb folder on a 1.44M diskette, then FDI.LIB must read the entire diskette and decompress all the data. This is about the worst access time possible. By contrast, if FOO.EXE were at the start of a folder (regardless of how large the folder is), then it would be read and decompressed with no extra overhead.

So, why would one not always Set FolderFileCountThreshold=1? Because doing so would reset the compression history after each file, resulting in a poor compression ratio. MakeCAB provides several variables and directives to provide very fine control over these issues:

Variable / Directive	More Compression; Slower Access Time	Less Compression; Faster Access Time
CabinetFileCountThreshold	Bigger numbers	Lower numbers
FolderFileCountThreshold	Bigger numbers	Lower numbers
FolderSizeThreshold	Bigger numbers	Lower numbers
MaxCabinetSize	Bigger numbers	Lower numbers
.New Folder	Don't use	Use often
.New Cabinet	Don't use	Use often

The MakeCAB defaults are configured for a floppy disk layout, with the assumption that the most common scenario is a full setup that will extract most of the files, so these are the settings:

Variable / Directive	Value
CabinetFileCountThreshold	2000 (Since we have to call FDICopy() on a cabinet and walk through all the FILE headers, we want this small enough so that isn't too much overhead, but large enough to keep the number of cabinets down.)
FolderFileCountThreshold	Unlimited (Let FolderSizeThreshold control folder size!)
FolderSizeThreshold	200K (Represents 600K-800K of source (assuming 3:1 or 4:1 compression ratio)
MaxCabinetSize	Unlimited (Let CabinetFileCountThreshold control the cabinet size!)

Of course, if you are tight for space on your CD-ROM, you'll probably boost the **FolderSizeThreshold** and **CompressionMemory** settings!

Back to: Top of page > MakeCAB Optimizing and Tuning

Piecemeal DDFs for Localization and Different Disk Sizes

MAKECAB.EXE was designed to minimize the amount of duplicate information needed to generate product layouts for different languages and disk sizes. A key feature is the ability to specify more than one DDF on the MAKECAB.EXE command line. For example:

acme.ddf	Some standard definitions to control the format of the output INF file
lang.ddf	Sets language-specific settings (SourceDir, for example)
disk.ddf	Sets the diskette sizes (CDROM, 1.2M, 1.44M, etc.)
product.ddf	Lists all the files in the product, and uses variables set in the previous DDFs to customize its operation

The following command line would be used to process this set of DDFs:

```
MakeCAB /f acme.ddf /f lang.ddf /f disk.ddf /f product.ddf
```

Back to: Top of page > MakeCAB Optimizing and Tuning

MakeCAB Concepts

The key feature of MakeCAB is that it takes a set of files and produces a *disk layout* while at the same time attempting to minimize the number of disks required. In order to understand how MakeCAB does this, three terms need to be defined: *cabinet*, *folder*, and *file*. MakeCAB takes all of the *files* in the product or application being compressed, lays the bytes down as one continuous byte stream, compresses the entire stream, chopping it up into *folders* as appropriate, and then fills up one or more *cabinets* with the *folders*.

Cabinet
: A normal file that contains pieces of one or more files, usually compressed. Also known as a "CAB file".

Folder
: A decompression boundary. Large folders enable higher compression, because the compressor can refer back to more data in finding patterns. However, to retrieve a file at the end of a folder, the entire folder must be decompressed. So there is a tradeoff between achieved compression and the quickness of random access to individual files.

File
: A file to be placed in the layout.

Back to: Top of page

Decoupling File Layout and INF Layout

MakeCAB has two "modes" for generating the INF file; *unified mode* and *relational mode*. In *unified mode*, the INF file is generated as file copy commands are processed in the DDF file. This is the default, and minimizes the amount of effort needed to construct a DDF file. However, this forces the INF file to list the files in the layout in exactly the same order as they are placed on disks/cabinets.

Example of a Unified DDF:

```
;** Set up INF formats before we do the disk layout, because MakeCAB
;   writes Disk and Cabinet information out as it is generated.
.OPTION EXPLICIT            ; Generate errors for undefined variables

.Set InfDiskHeader="[disk list]"
.Set InfDiskHeader1=";<disk number>,<disk label>"
.Set InfDiskLineFormat="*disk#*,*label*"

.Set InfCabinetHeader="[cabinet list]"
.Set InfCabinetHeader1=";<cabinet number>,<disk number>,<cabinet file name>"
.Set InfCabinetLineFormat="*cab#*,*disk#*,*cabfile*"

.Set InfFileHeader=";*** File List ***"
.Set InfFileHeader1=";<disk number>,<cabinet number>,<filename>,<size>"
.Set InfFileHeader2=";Note: File is not in a cabinet if cab# is 0"
.Set InfFileHeader3=""
.Set InfFileLineFormat="*disk#*,*cab#*,*file*,*date*,*size*"


.set GenerateInf=ON         ; Unified mode - create the INF file as we go

;** Setup files.  These don't need to be in the INF file, so we put
;   /inf=NO on these lines so that MakeCAB won't generate an error when
;   it finds that these files are not mentioned in the INF portion of
;   the DDF.

.set Compress=OFF
.set Cabinet=OFF
setup.exe /inf=NO           ; This file doesn't show up in INF
setup.inf /inf=NO           ; This file doesn't show up in INF

;** Files in cabinets
.set Compress=ON
.set Cabinet=ON

;* Put all bitmaps together to help compression
a1.bmp                      ; Bitmap for client1.exe
b1.bmp                      ; Bitmap for client1.exe
c1.bmp                      ; Bitmap for client1.exe
d1.bmp                      ; Bitmap for client1.exe
a2.bmp                      ; Bitmap for client1.exe
b2.bmp                      ; Bitmap for client2.exe
c2.bmp                      ; Bitmap for client2.exe
d2.bmp                      ; Bitmap for client2.exe
shared.dll  /date=10/12/93  ; File needed by client1.exe and client2.exe
client1.exe                 ; needs shared.dll
client2.exe                 ; needs shared.dll

;*** The End
```

In *relational mode* the DDF has *file reference* lines to specify the exact placement of file information lines, including the ability to list the same file multiple times. This feature is important for INF structures which use section headers (e.g. "[clipart]", "[screen savers]") to identify sets of files for particular functionality, and for which the same file may need to be included in more than one section. For example, a product may have several optional features, all of which require a DLL file named "shared.dll". Rather than having "shared.dll" stored multiple times (once for each section which uses the file), a waste of disk space, a single copy of the file can be stored, and then referenced by all of the sections which require it.

A relational mode DDF is similar to a unified mode DDF, with the exception that a ".set GenerateInf=OFF" line must be inserted before the product's files are listed (as shown below). Once all of the files have been listed, the INF file generating portion of the DDF begins, and a ".set GenerateInf=ON" line must be inserted, followed by the section definitions.

Example of a Relational DDF:

```
    ;** Set up INF formats before we do the disk layout, because MakeCAB
    ;   writes Disk and Cabinet information out as it is generated.
    .OPTION EXPLICIT            ; Generate errors for undefined variables

    .Set InfDiskHeader="[disk list]"
    .Set InfDiskHeader1=";<disk number>,<disk label>"
    .Set InfDiskLineFormat="*disk#*,*label*"

    .Set InfCabinetHeader="[cabinet list]"
    .Set InfCabinetHeader1=";<cabinet number>,<disk number>,<cabinet file name>"
    .Set InfCabinetLineFormat="*cab#*,*disk#*,*cabfile*"

    .Set InfFileHeader=";*** File List ***"
    .Set InfFileHeader1=";<disk number>,<cabinet number>,<filename>,<size>"
    .Set InfFileHeader2=";Note: File is not in a cabinet if cab# is 0"
    .Set InfFileHeader3=""
    .Set InfFileLineFormat="*disk#*,*cab#*,*file*,*date*,*size*"


;
; *** Here is where we list all the files
;
    .set GenerateInf=OFF        ; RELATIONAL MODE - Do disk layout first

    ;** Setup files.  These don't need to be in the INF file, so we put
    ;   /inf=NO on these lines so that MakeCAB won't generate an error when
    ;   it finds that these files are not mentioned in the INF portion of
    ;   the DDF.

    .set Compress=OFF
    .set Cabinet=OFF
    setup.exe /inf=NO           ; This file doesn't show up in INF
    setup.inf /inf=NO           ; This file doesn't show up in INF

    ;** Files in cabinets
    ;
    .set Compress=ON
    .set Cabinet=ON

    ;* Put all bitmaps together to help compression
    a1.bmp                      ; Bitmap for client1.exe
    b1.bmp                      ; Bitmap for client1.exe
    c1.bmp                      ; Bitmap for client1.exe
    d1.bmp                      ; Bitmap for client1.exe
    a2.bmp                      ; Bitmap for client1.exe
    b2.bmp                      ; Bitmap for client2.exe
    c2.bmp                      ; Bitmap for client2.exe
    d2.bmp                      ; Bitmap for client2.exe
    shared.dll  /date=10/12/93  ; File needed by client1.exe and client2.exe
    client1.exe                 ; needs shared.dll
    client2.exe                 ; needs shared.dll


;
; *** Now we're generating the INF file
;
    .set GenerateInf=ON

    ;** Feature One files
    .InfBegin File
    [feature One]
    ;Files for feature one
    .InfEnd
    client1.exe
    shared.dll  /date=04/01/94  ; Override date
    a1.bmp
    b1.bmp
    c1.bmp
    d1.bmp

    ;** Feature Two files
    .InfBegin File

    [feature Two]
    ;Files for feature Two
    ;Note that shared.dll is also required by Feature One
    .InfEnd
    client1.exe
    shared.dll
    a2.bmp
    b2.bmp
    c2.bmp
    d2.bmp

    ;*** The End
```

The generated INF file would look something like this:

```
[disk list]
;<disk number>,<disk label>
1,"Disk 1"

[cabinet list]
;<cabinet number>,<disk number>,<cabinet file name>
1,1,cabinet.1

;*** File List ***
;<disk number>,<cabinet number>,<filename>,<size>
;Note: File is not in a cabinet if cab# is 0

[feature One]
;Files for feature one
1,1,client1.exe,12/12/93,1234
1,1,shared.dll,04/01/94,1234
1,1,a1.bmp,12/12/93,573
1,1,b1.bmp,12/12/93,573
1,1,c1.bmp,12/12/93,573
1,1,d1.bmp,12/12/93,573
```

```
[feature Two]
;Files for feature Two
;Note that shared.dll is also required by Feature One
1,1,client1.exe,12/12/93,1234
1,1,shared.dll,10/12/93,1234
1,1,a2.bmp,12/12/93,643
1,1,b2.bmp,12/12/93,643
1,1,c2.bmp,12/12/93,643
1,1,d2.bmp,12/12/93,643
```

Notes:

1. In "relational" mode, only the last setting of a particular **Inf**Xxx default parameter variable (both standard parameters like **InfDate**, **InfTime**, etc. and custom parameters) in the layout portion (i.e. the first part) of the DDF is respected.

   Example:

   If you did ".set **InfDate**=12/05/92" at the start of the layout portion, and then did ".set **InfDate**=01/01/94" in the middle of the layout portion, the latter value would be used for the entire INF file.

2. Any parameters on a reference line will override parameters on the corresponding file copy line.

   Example:

   ```
   ;* layout portion
   bar /x=1

   ;* INF portion
   bar /x=2            ; INF file will have value 2
   ```

3. In "relational" mode, each file copy command in the layout portion of the DDF must be referenced at least once in a reference command in the INF portion of the DDF. Any files that are not referenced will cause an error during pass 1. The **/inf=no** parameter must be specified on any file copy commands for files which are going to be omitted from the INF file (such as SETUP.EXE and SETUP.INF).
4. In "relational" mode, **UniqueFiles** must be **ON**, because the destination file name is used in the INF portion of the DDF to refer back to file information.

Back to: Top of page > MakeCAB Concepts

MAKECAB.EXE

MAKECAB.EXE is designed to produce the final distribution files and cabinets for an entire product in a single run. The most common way to use MAKECAB.EXE is to supply a *directives file* that controls how files are compressed and stored into one or more cabinets.

This section includes:

MAKECAB.EXE Syntax

MAKECAB.EXE Directive File Syntax

Back to: Top of page

MAKECAB.EXE Syntax

There two primary forms of MAKECAB.EXE usage. The first is used for compressing a single file, while the second is used for compressing multiple files.

```
MAKECAB  [/Vn] [/D variable=value ...] [/L directory] source [destination]
MAKECAB  [/Vn] [/D variable=value ] /F directives_file [...]
```

The parameters are described below.

Parameter	Description
source	A file to be compressed.
destination	The name of the file to receive the compressed version of the source file. If not supplied, a default destination name is constructed from the source file name according to the rules defined by the CompressedFileExtensionChar variable. You can use /D CompressedFileExtensionChar=c on the command line to change the appended character.
/D variable=value	Set variable to be equal to value. Equivalent to using the **.Set** command in the directives file. For example, a single directive file could be used to produce layouts for different disk sizes by running MakeCAB once with different values of **MaxDiskSize** defined: /D MaxDiskSize=1.44M. Both standard MakeCAB variables and custom variables may be defined in this way. If **.Option Explicit** is specified in a directive file, then variable must be defined with a **.Define** command in a directive file.
/L directory	Specifies an output directory where the compressed file will be placed (most useful when destination is not supplied).
/F directives_file	A file containing commands for MAKECAB.EXE to execute. If more than one directive file is specified (/F file1 /F file2 ...), they are processed in the order (left to right) specified on the command line. Variable settings, open cabinets, open disks, etc. are all carried forward from one directive file to the next (just as if all of the files had been concatenated together and presented as a single file to MakeCAB). For example, this is intended to simplify the work for a product shipped in multiple languages. There would be a short, language-specific directives file, and then a single, large master directives file that covers the bulk of the product.
/Vn	Set debugging verbosity level (0=none,...,3=full)

Back to: Top of page > MAKECAB.EXE

MAKECAB.EXE Directive File Syntax

Before diving into the details of the syntax of the directives file, provided here is an example of what the Excel directives file might look like:

```
;*** EXCEL MAKECAB Directive file
;
.Set DiskLabel1=Setup               ; Label of first disk
.Set DiskLabel2=Program             ; Label of second disk
.Set DiskLabel3="Program Continued"  ; Label of third disk
.Set CabinetNameTemplate=EXCEL*.CAB  ; EXCEL1.CAB, EXCEL2.CAB, etc.
.Set MaxDiskSize=1.44M              ; 3.5" disks

;** Setup.exe and setup.inf are placed uncompressed in the first disk
.Set Cabinet=off
.Set Compress=off
bin\setup.exe                       ; Just copy SETUP.EXE as is
bin\setup.inf                       ; Just copy SETUP.INF as is
;** The rest of the files are stored, compressed, in cabinet files
.Set Cabinet=on
.Set Compress=on
bin\excel.exe                       ; Big EXE, will span cabinets
bin\excel.hlp
bin\olecli.dll
bin\olesrv.dll
...
```

Here are some additional notes on the general syntax and behavior of MakeCAB Directive Files:

1. MakeCAB will place files on disks (and in cabinets) in the order they are specified in the directive file(s).
2. When ever a filename or directory is called for, you may supply either a relative (e.g., foo\bar, ..\foo) or an absolute (e.g., c:\banana, x:\slm\src\bin) path.
3. Optimal compression is achieved when files with similar types of data are grouped together.
4. MakeCAB is controlled in large part by setting *variables*. MakeCAB has many predefined variables, all of which have default values chosen to represent the most common case. You can modify these variables, and you can define your own variables as well.
5. The value of a variable is retrieved by enclosing the variable name in percent (%) signs. If the variable is not defined, an error is generated. If you want an explicit percent sign, use two adjacent percent signs (%%). MakeCAB will collapse this to a single percent sign (%).
6. Variable substitution is only done once. For example, .Set A=One [A is "One"]; .Set B=%%A%% (B is "%A%"); .Set C=%B% (C is "%A%", **not** "One").
7. Variable substitution is done before any other line parsing, so variables can be used anywhere.
8. Variables values may include blanks. Quote (") or apostrophe(') marks may be used in .Set statements to capture blanks. If you want an explicit quote(") or apostrophe('), you can intermix these two marks (use one for bracketing so that you may specify the other), or, as with the percent sign above, you can specify two adjacent marks ("") and MakeCAB will collapse this to a single mark(").
9. All sizes are specified in bytes.
10. There are a few special values for common disks sizes (CDROM, 1.44M, 1.2M, 720K, 360K) that can be used for any of the predefined MakeCAB variables that describe the attributes of a disk (**MaxDiskSize**, **ClusterSize**, **MaxDiskFileCount**). MakeCAB has built-in knowledge about the correct values of these attributes for these common disk sizes.
11. MakeCAB does not check for 8.3 filename limitations directly, but rather depends upon the underlying operating system to do filename validity checking (this will allow MakeCAB to work with long file names.)
12. MakeCAB makes two passes of the directive file(s). On the first pass, MakeCAB checks for syntax errors and makes sure that all of the files can be found. This is very fast, and reduces the chance that the second pass, where the actual data compression occurs, will have any problems. This is important because compression is very time consuming, so MakeCAB wants to avoid, for example, spending an hour compressing files only to find that a file toward the end of the directive file(s) cannot be found.

Back to: Top of page > MAKECAB.EXE

This section includes:

Command Summary

Variable Summary

InfDisk/Cabinet/FileLineFormat Syntax and Semantics

INF Parameters

Command Details

Variable Details

Command Summary

The following table provides a summary of the MakeCAB Directive File syntax. Directives begin with a period ("."), followed by a command name, and possibly by blank delimited arguments. Note that a File Copy command is distinguished from a File Reference command by the setting of the **GenerateInf** variable.

Syntax	Description
;	Comment (anywhere on a DDF line)
src [dest] [**/inf=yes|no**] [**/unique=yes|no**] [/x=y ...]	File Copy command
dest [/x=y ...]	File Reference command
.Define variable=[value]	Define *variable* to be equal to *value* (see .Option Explicit)
.Delete variable	Delete a variable definition
.Dump	Display all variable definitions
.InfBegin Disk | Cabinet | Folder	Copy lines to specified INF file section
.InfEnd	End an **.InfBegin** section

.InfWrite string	Write "string" to file section of INF file
.InfWriteCabinet string	Write "string" to cabinet section of INF file
.InfWriteDisk string	Write "string" to disk section of INF file
.New Disk | Cabinet | Folder	Start a new Disk, Cabinet, or Folder
.Option Explicit	Require **.Define** first time for user-defined variables
.Set variable=[value]	Set *variable* to be equal to *value*
%variable%	Substitute value of *variable*
<blank line>	Blank lines are ignored

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax

Variable Summary

Standard Variables	**Description**
Cabinet=**ON** | **OFF**	Turns Cabinet Mode on or off
CabinetFileCountThreshold=count	Threshold count of files per Cabinet
CabinetName*n*=filename	Cabinet file name for cabinet number *n*
CabinetNameTemplate=template	Cabinet file name template; * is replaced by Cabinet number
ChecksumWidth=**1** | **2** | ... | **8**	Max low-order hex digits displayed by INF **csum** parameter
ClusterSize=bytesPerCluster	Cluster size on diskette (default is 512 bytes)
Compress=**ON** | **OFF**	Turns compression on or off
CompressedFileExtensionChar=char	Last character of the file extension for compressed files
CompressionType=**MSZIP**	Compression engine
DestinationDir=path	Default path for destination files (stored in cabinet file)
DiskDirectory*n*=directory	Output directory name for disk *n*
DiskDirectoryTemplate=template	Output directory name template; * is replaced by disk number
DiskLabel*n*=label	Printed disk label name for disk *n*
DiskLabelTemplate=template	Printed disk label name template; * is replaced by disk number
DoNotCopyFiles= **ON** | **OFF**	Controls whether files are actually copied (ACME ADMIN.INF)
FolderFileCountThreshold=count	Threshold count of files per Folder
FolderSizeThreshold=size	Threshold folder size for current folder
GenerateInf=**ON** | **OFF**	Control Unified vs. Relation INF generation mode
Inf*Xxx*=string	Set default value for INF Parameter Xxx
InfCabinetHeader[*n*]=string	INF cabinet section header text
InfCabinetLineFormat[*n*]=format string	INF cabinet section detail line format
InfCommentString=string	INF comment string
InfDateFormat=**yyyy-mm-dd** | **mm/dd/yy**	INF date format
InfDiskHeader[*n*]=string	INF disk section header text
InfDiskLineFormat[*n*]=format string	INF disk section detail line format
InfFileHeader[*n*]=string	INF file section header text
InfFileLineFormat[*n*]=format string	INF file section detail line format
InfFileName=filename	Name of INF file
InfFooter[*n*]=string	INF footer text
InfHeader[*n*]=string	INF header text
InfSectionOrder=[**D** | **C** | **F**]*	INF section order (disk, cabinet, file)
MaxCabinetSize=size	Maximum cabinet file size for current cabinet
MaxDiskFileCount=count	Maximum count of files per Disk
MaxDiskSize[*n*]=size	Maximum disk size
MaxErrors=count	Maximum errors allowed before pass 1 terminates
ReservePerCabinetSize=size	Base amount of space to reserve for FCRESERVE data
ReservePerDataBlockSize=size	Amount of space to reserve in each data block
ReservePerFolderSize=size	Amount of additional space in FCRESERVE for each folder
RptFileName=filename	Name of RPT file
SourceDir=path	Default path for source files
UniqueFiles=**ON | OFF**	Control whether duplicate desintation file names are allowed
Cabinet=**ON** | **OFF**	Turns Cabinet Mode on or off

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax

InfDisk/Cabinet/FileLineFormat Syntax and Semantics

The **InfDiskLineFormat**, **InfCabinetLineFormat**, and **InfFileLineFormat** variables are used to control the formatting of the "detail" lines in the INF file. The syntax of the values assigned to these variables is as follows:

1. The "*" character is used to bracket replaceable parameters.
2. Two "*" characters in a row ("**") are replaced by a single "*".
3. A replaceable parameter name may be one of the standard ones defined by MakeCAB, or it may be a custom parameter. The value used for a parameter is found in the following order:
    a. If a parameter is specified on a File Copy or File Reference command, the specified value is used.
    b. If a variable InfXxxx is defined for this parameter, its value is used. The parameter is a standard parameter, and its defined value is used.
4. Braces "{}" may be used to indicate portions of text plus exactly one parameter that are omitted if the parameter value is blank.

For example, "{*id*,}*file*,*size*" will generate the following strings, depending upon the values of id, file, and size:

id	file	size	Output String
	foo.dat	23	foo.dat,23
17	foo.dat	23	17,foo.dat,23
17		23	17,,23

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax

INF Parameters

The following table lists the standard parameters that may be specified in INF line formats and on File Copy and File Reference commands. The Disk, Cab, and File columns indicate which parameters are supported in the **InfDiskLineFormat**, **InfCabinetLineFormat**, and **InfFileLineFormat**, respectively. In addition, the File column also indicates which parameters may be specified on the File Copy and File Reference commands.

Parameter	Disk	Cab	File	Description
attr			Yes	File attributes (A=archive, R=read-only, H=hidden, S=system)
cab#		Yes	Yes	Cabinet number (0 means not in cabinet, 1 or higher is cabinet number)
cabfile		Yes		Cabinet file name
csum			Yes	Checksum
date			Yes	File date (mm/dd/yy or yyyy-mm-dd, depending upon **InfDateFormat**)
disk#	Yes	Yes	Yes	Disk number (1-based)
file			Yes	Destination file name in layout (in cabinet or on a disk)
file#			Yes	Destination file number in layout (first file is 1, second file is 2, ...); the order of File Copy Commands controls the file number, so in relational INF mode the order of File Reference Commands has no affect on the file number.
label	Yes			Disk user-readable label (value comes from **DiskLabel***n*, if defined, and otherwise is constructed from **DiskLabelTemplate**).
lang			Yes	Language (i.e., VER.DLL info) in base 10, blank separated if multiple values
size			Yes	File size (only affects value written to INF file)
time			Yes	File time (hh:mm:ss[a|p])
ver			Yes	Binary File version (n.n.n.n base 10 format)
vers			Yes	String File version -- can be different from **ver**!
attr			Yes	File attributes (A=archive, R=read-only, H=hidden, S=system)

Just as custom INF parameters can be defined by using the **.Define** and **.Set** command (e.g., **.Set** InfCustom=default value), the **.Set** command can also be used to override the values of these parameters. This is most obviously useful for the **date** and **time** parameters, as it provides a simple way to "date stamp" all the files in a layout; and for the **attr** parameter, this provides a way to force a consistent set of file attributes (commonly used to clear the read-only and archive attribute bits).

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax

Command Details

;

A comment line.
A comment may appear anywhere in a directive file. In addition, any line may include a comment at the end. Any text on the line following the comment is ignored.

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Command Summary

source [destination] [**/INF= YES** | **NO**] [**/UNIQUE=YES** | **NO**] [**/x=y** [**/x=y** ...]]

A File Copy Command; specifies a file to be placed onto a disk or cabinet. If **GenerateInf** is **OFF**, then lines without leading periods are interpreted as File Copy Commands.

source is a file name, and may include a relative or absolute path specification. The **SourceDir** variable is applied first, if specified.

destination is the name to store in the cabinet file (if **Cabinet** is **On**), or the name for the destination file (if **Cabinet** is **Off**). The **DestinationDir** variable is used as a prefix. **/INF**=**YES** | **NO** controls whether *destination* must specified in a Reference command in the INF section of the DDF. If **YES** is specified (the default), then *destination* must be specified in at least one Reference command. If **NO** is specified, then *destination* does not have to be specified in any Reference command. This parameter is used only if Relational INF mode is selected (see the **GenerateInf** variable), as Unified mode does not support Reference commands.

/UNIQUE=**YES** | **NO** controls whether *destination* must be unique throughout the layout. Specifying this parameter on the file copy command overrides the default setting controlled by the **UniqueFiles** variable (which defaults to YES). If Relational INF mode is selected (see the **GenerateInf** variable), then **UniqueFiles** must be **YES**.

/x=**y** permits standard and custom INF parameters to be applied to a file copy command. These parameters are carried along with the file by MakeCAB and used to format file detail lines in the INF file. In addition, the **/Date**, **/Time**, and **/Attr** parameters also control the values that are placed in the cabinet files or on the disk layout (for files outside of a cabinet). This permits a great deal of flexibility in customizing the INF file format. A parameter "x" is defined to have the value "y" (which may be empty). Quotes can be used in "y" to include blanks or other special characters. If a parameter "x" is also defined on a File Reference command, that setting overrides any setting for "x" specified on the referred to File Copy command. See INF Parameters for a list of standard parameters.

NOTE: You must define a variable InfX if you are going to use /X=y on a File Copy (or File Reference) command. If no such variable is defined, then /X=y will generate an error. This behavior ensures that there is a default value for every parameter, and makes it easier to catch inadvertent typing errors.

If the *destination* is not specified, its default value depends upon the **Cabinet** and **Compress** variables, as indicated by the following table, using BIN\EXCEL.EXE as a sample source file name. Note that the variable **CompressedFileExtensionChar** controls the actual character

used to indicate a compressed file. Note also that the **DestinationDir** variable is prefixed to the destination name before it is stored in the cabinet file.

	Compress = OFF	**Compress = ON**
Cabinet = OFF	EXCEL.EXE -- uncompressed, not in a cabinet.	EXCEL.EX_ -- compressed, not in cabinet (actually, this is a cabinet with a single file! -- See note below.)
Cabinet = ON	EXCEL.EXE -- uncompressed, in a cabinet.	EXCEL.EXE -- compressed, in a cabinet

NOTE: Compressing a single file is generally not a good idea, as better compression is achieved by compressing across file boundaries (hence cabinet files). However, MakeCAB supports this in case clients used to the old way of writing a setup program need this feature. Instead of having two different file formats, though, we simple create a cabinet that has just the one file in it.

Examples:

```
.Set Compress=OFF             ; Turn off compression
.Set Cabinet=OFF              ; No cabinet file
setup.exe /inf=no             ; Setup is put on disk 1, won't be in INF
setup.inf                     ; Classic chicken & the egg problem

.Set Compress=ON              ; Turn compression on
readme.txt                    ; Placed on disk 1 as README.TX_
.Set Cabinet=ON               ; Turn cabinet file creation on
bin\excel.exe                 ; Placed in cabinet as EXCEL.EXE
msdraw.exe msapps\msdraw.exe  ; Placed in cabinet as MSAPPS\MSDRAW.EXE
a.txt dup.txt /unique=no      ; Another dup.txt is allowed
b.txt dup.txt /unique=no      ; And here it is
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Command Summary

destination [**/x=y** [**/x=y** ...]]

A File Reference Command; specifies that information for a file (previously specified in a File Copy command) is to be written to the File section of the INF file. This command is only supported in Relational INF mode. If **GenerateInf** is **ON**, then lines without leading periods are interpreted as File Reference Commands.

destination is the name of a file previously specified in a File Copy command as the **destination** in the layout (not the source!). Therefore, **UniqueFiles** is required to be **ON**.

/x=y permits standard and custom INF parameters to be applied to a file reference command. These parameters are merged with any parameters specified on the referenced File Copy command, with parameters on the File Reference command taking precedence.

A parameter "x" is defined to have the value "y" (which may be empty). Quotes can be used in "y" to include blanks or other special characters. . See INF Parameters for a list of standard parameters.

NOTE: You must define a variable InfX if you are going to use /X=y on a File Reference (or File Copy) command. If no such variable is defined, then /X=y will generate an error. This behavior ensures that there is a default value for every parameter, and makes it easier to catch inadvertent typing errors.

Examples:

```
.Set GenerateInf=OFF     ; Relational INF mode; file layout
setup.exe /inf=no        ; Setup is put on disk 1, won't be in INF
readme.txt
shared.dll /special=yes  ; Custom parameter

.Set GenerateInf=ON      ; INF section of DDF
.InfWrite [Common]
readme.txt
.InfWrite [One]
shared.dll /special=no   ; Override parm on file copy command
.InfWrite [Two]
shared.dll               ; Use /special value from file copy
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Command Summary

.Define variable=[value]

Define *variable* to be equal to *value*.

To use *variable*, surround it with percent signs (%) -- %variable%.

Using an undefined variable is an error, and will cause MakeCAB to stop before pass 2.

value may include references to other variables.

Leading and trailing blanks in *value* are discarded.

Blanks may be enclose in quote (") or apostrophe (') marks.

Explicit percent signs (%), quotes ("), or apostrophes (') must be specified twice.

NOTE: If **.Option Explicit** is specified, then you must first use **.Define** to define any user-defined variables before you can use **.Set** to modify them. For standard MakeCAB variables, **.Define** is not permitted, and only **.Set** may be used on. If **.Option Explicit** is not specified, then **.Define** is equivalent to **.Set**.

Examples:

```
.Define lang=ENGLISH               ; Set language
.Define country=USA                ; Set country
.Define SourceDir=%lang%\%country%  ; SourceDir = [ENGLISH\USA]
.Define join=%lang%%country%       ; join = [ENGLISHUSA]
.Define success=100%%              ; success = [100%]
.Define SourceDir=                 ; SourceDir = []
.Define contraction="don't"        ; contraction = [don't]
```

```
.Define contraction=don''t          ; contraction = [don't]
.Define someSpaces=  hi there       ; someSpaces = [hi there]
.Define someMore="  blue dog  "     ; someMore = [  blue dog  ]
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Command Summary

.Delete variable

Delete a variable definition.

You may only delete variables that have been created by **.Define** or **.Set** commands. Standard MakeCAB variables may not be deleted.

Examples:

```
.Set myVariable=raisin
.Delete myVariable      ; Delete myVariable
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Command Summary

.Dump

Display the entire MakeCAB variable table.

This command can be used to aid debugging of complicated (or not so complicated) MakeCAB directive files. Note that the dump will be displayed during pass 1 and again during pass 2.

Examples:

```
.Dump               ; Dump variable table to stdout
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Command Summary

.InfBegin DISK | CABINET | FILE

Start a block of one or more lines to write to the specified area of the INF file.

The lines in the block will be copied unmodified to the specified section of the INF file, so no MakeCAB variable substitution will be performed. Similarly, MakeCAB will not strip comments.

Use **.InfWrite**, **.InfWriteCabinet**, or **.InfWriteDisk** if you need variable substitution.

Examples:

```
.InfBegin disk               ; Text for disk section of INF file
;This is a comment for the disk section.  MakeCAB will not process
;this line, so, for example, %var% will not be substituted.
.InfEnd
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Command Summary

.InfEnd

Terminate an .InfBegin block.

Examples:

```
.InfEnd            ; Close an .InfBegin block
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Command Summary

.InfWrite string

Write *string* to the file area of the INF file.

Note that lines will have MakeCAB comments removed and variable values substituted. If you want to avoid this processing, use the **.InfBegin File** command. Leading whitespace is normally removed, but you can override this by placing whitespace in quotes (see examples below)

Examples:

```
.InfWrite [A Section Header]  ; Text for file section, this comment
                              ;    will not appear.

.InfWrite ;<disk>,<file>      ; MakeCAB strips off the comments, so this
                              ;    command just writes a blank line!

.InfWrite ";<disk>,<file>"    ; Get that comment in the INF file

.InfWrite "  "%someVar%       ; Get leading space on the INF line
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Command Summary

.InfWriteCabinet string

Write *string* to the cabinet area of the INF file.

Note that lines will have MakeCAB comments removed and variable values substituted. If you want to avoid this processing, use the **.InfBegin Cabinet** command.

Examples:

```
.InfWriteCabinet 40%% off your favorite furniture  ; %% collapse down to
                    ; one %, because MakeCAB does variable
                    ; substitution on the string.
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Command Summary

.InfWriteDisk string

Write *string* to the disk area of the INF file.

Note that lines will have MakeCAB comments removed and variable values substituted. If you want to avoid this processing, use the **.InfBegin Disk** command.

Examples:

```
.InfWriteDisk The Rain in Spain falls Mainly on the Plain
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Command Summary

.New Disk | Cabinet | Folder

Force a disk, cabinet, or folder break.

This is used to complete the current disk, cabinet, or folder, and start a new one.

Examples:

```
.New Disk     ; Start a new disk
.New Cabinet  ; Start a new cabinet
.New Folder   ; Start a new folder
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Command Summary

.Set variable**=**value

Set *variable* to be equal to *value*.

To use *variable*, surround it with percent signs (%) -- %variable%.

Using an undefined variable is an error, and will cause MakeCAB to stop before pass 2.

value may include references to other variables.

value may be empty, in which case *variable* is set to the empty string.

Leading and trailing blanks in *value* are discarded.

Blanks may be enclose in quote (") or apostrophe (') marks.

Explicit percent signs (%), quotes ("), or apostrophes (') must be specified twice.

NOTE: If **.Option Explicit** is specified, then you must first use **.Define** to define any user-defined variables before you can use **.Set** to modify them. For standard MakeCAB variables, **.Define** is not permitted, and only **.Set** may be used on.

Examples:

```
.Set lang=ENGLISH               ; Set language
.Set country=USA                ; Set country
.Set SourceDir=%lang%\%country% ; SourceDir = [ENGLISH\USA]
.Set join=%lang%%country%       ; join = [ENGLISHUSA]
.Set success=100%%              ; success = [100%]
.Set SourceDir=                 ; SourceDir = []
.Set contraction="don't"        ; contraction = [don't]
.Set contraction=don''t         ; contraction = [don't]
.Set someSpaces=  hi there      ; someSpaces = [hi there]
.Set someMore="  blue dog  "    ; someMore = [  blue dog  ]
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Command Summary

Variable Details

The standard MakeCAB variables are listed below. These variables are predefined, and each of them have default value, which is used if you do not set the variable from the command line (/D var=value) or prior to the time you explicitly set the variable with a **.Define** or **.Set** command in a directive file.

You can create your own variables as well, using the **.Define** command if you specify **.Option Explicit**, and the **.Set** command otherwise.

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax

Cabinet=On | Off

Turns *cabinet mode* on or off.

Default: .Set Cabinet=On ; Cabinet mode is ON

When cabinet mode is **On**, the following applies:

a. Files are stored in a cabinet, whose name is taken from the CabinetNameTemplate variable

b. If the compressed size of a file would cause the current Cabinet to exceed the current MaxCabinetSize variable, then as much of the compressed file as possible is stored in the current Cabinet, that Cabinet is closed, and a new Cabinet is created. Note that it is possible for a large file to span multiple Cabinets!

c. If the compressed size of a file (or set of files, if the files are small) would cause the current Folder to exceed the current MinFolderSize variable, these files are the last ones added to the current Folder, a new Folder is started for any subsequent files. (See note below.) Note that if the current Folder cannot fit in the current Cabinet, as much as possible of the Folder is stored in the current Cabinet, and the remainder of the Folder is stored in the next Cabinet. This means that it is possible for several files to be continued from one Cabinet file to the next Cabinet file!

NOTE: The motivation here is that a Folder is a decompression boundary, and so is advisory. To access a file in a Folder, you must start decompressing from the beginning of a Folder, potentially decompressing (and discarding) many files until you arrive at the desired file. If we made the current folder larger, then this file just added would take longer to access. In general, the **MinFolderSize** variable should be several times larger than 32K, to be of any utility.

When cabinet mode is **Off**, the following applies:

a. Files are stored in individual files

b. If the destination file is not supplied, the default name is controlled by the *compression mode* (see the **Compress** variable)

Examples:

```
.Set Cabinet=OFF   ; Files not in cabinets...
.Set Compress=OFF  ; ...and no compression.
setup.exe          ; Setup program is simply copied to disk.
.Set Cabinet=ON    ; Use a cabinet...
.SET Compress=ON   ; ...and compress remaining files.
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

CabinetFileCountThreshold=count

Sets a goal for the maximum number of files in a cabinet.

Default: .Set CabinetFileCountThreshold=0 ; Default is no threshold

count is a threshold for the number of files to store in a cabinet. Once this count has been reached, MakeCAB will close the current cabinet as soon as possible. Due to the blocking of files for compression purposes, it is possible that the cabinet will contain more files than specified by this variable.

If *count* is 0, then there is no limit on the number files per cabinet.

Examples:

```
.Set CabinetFileCountThreshold=100  ; Shoot for 100 files per cabinet
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

CabinetNamen=filename

The cabinet file name for the specified cabinet.

Default: ; By default none of these variables are defined

If this variable is not defined for a particular disk, then MakeCAB uses the **CabinetNameTemplate** to construct the cabinet name.

Examples:

```
.Set CabinetName1=one.cab
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

CabinetNameTemplate=template

Sets the cabinet file name template.

Default: .Set CabinetNameTemplate=*.CAB ; 1.CAB, 2.CAB, ...

This template is used to construct the file name of each cabinet. The "*" in this template is replaced by the cabinet number (1, 2, etc.). This variable is used only if no variable **CabinetName*n*** exists for cabinet *n*.

NOTE: Be sure that the expanded cabinet name does not exceed the limits for your file system! For example, if you used "CABINET*.CAB", and MakeCAB had to create 10 or more cabinets, then you would have cabinet names like CABINET10.CAB, which is 9.3, which is an invalid name in the FAT file system. Unfortunately, MakeCAB would not detect this until it had already created 9 cabinets!

Examples:

```
.Set CabinetNameTemplate=EXCEL*.DIA  ; EXCEL1.DIA, EXCEL2.DIA, etc.
.Set CabinetNameTemplate=*.          ; 1, 2, 3, etc.
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

ChecksumWidth=1 | **2** | ... | **8**

Sets the maximum number of low-order hex digits displayed by **InfFileLineFormat csum** parameter.

Default: .Set ChecksumWidth=8 ; Default is all 8 hex digits (csum is a 32-bit value)

The presence of the **csum** parameter in the **InfFileLineFormat** variable causes MakeCAB to compute a 32-bit CRC for each file and write that checksum to the INF file. While leading zeros are not written out, the presence of these checksums can significantly increase the size of the INF file. You can use **ChecksumWidth** to restrict the size of the checksum written to the INF file. If a value less than 8 is specified, then MakeCAB will mask off the high-order bits of the 32-bit checksum to produce a value for the INF file that is at most the number of hex digits specified.

Examples:

```
.Set ChecksumWidth=4  ; Only display the low order 4 hex digits
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

ClusterSize=bytesPerCluster
The cluster size of the distribution media.

Default: .Set ClusterSize=512 ; 1.44M and 1.2M floppies have 512-byte clusters

This is used by MakeCAB to round up the sizes of files and cabinets to a cluster boundary, so it can determine when to switch to the next disk.

You can use a standard disk size from the following list, and MakeCAB will supply the known cluster size for that disk size:

- **1.44M**
- **1.25M** (Japanese NEC 3.5" drive capacity)
- **1.2M**
- **720K**
- **360K**
- **CDROM**

Examples:

```
.Set ClusterSize=1.44M  ; Use known 1.44M floppy info
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

Compress=ON | **OFF**
Turn file compression on or off.

Default: .Set Compress=On ; Compression is on

While compression is usually on, you generally turn if off for the first few files on disk 1 (SETUP.EXE, for example). This applies regardless of the **Cabinet** setting, so it is valid to store one or more uncompressed files in a Cabinet File.

Examples:

```
.Set Cabinet=OFF   ; Files not in cabinets...
.Set Compress=OFF  ; ...and no compression.
setup.exe          ; Setup program is simply copied to disk.
.Set Cabinet=ON    ; Use a cabinet...
.SET Compress=ON   ; ...and compress remaining files.
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

CompressedFileExtensionChar=char
Last character in file name used when compressing an individual file.

Default: .Set CompressedFileExtensionChar=_ ; Default is an underscore ("_")

If **Cabinet**=**OFF** and **Compress**=**ON** , then MakeCAB will compress an individual file. While the compressed files is stored in a Cabinet File, it has only a single file. To maintain some consistency with existing setup compression products, the default compressed file name is constructed by taking the source file name and replacing the last character of the file extension with the setting of this variable.

Examples:

```
.Set CompressedFileExtensionChar=$  ; SAMPLE.EXE => SAMPLE.EX$
                                    ; SAMPLE.EX  => SAMPLE.EX$
                                    ; SAMPLE.E   => SAMPLE.E$
                                    ; SAMPLE.    => SAMPLE.$
                                    ; SAMPLE     => SAMPLE.$
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

CompressionType=MSZIP
Select compression engine.

Default: .Set CompressionType=MSZIP ; Default is MSZIP compressor

MSZIP is the default compression type supported by Microsoft. This version of MakeCAB.EXE also supports the LZX compression method, which can achieve higher compressions ratios.

Using MSZIP compression and FolderSizeThreshold=1 will generate a cabinet file approximately the same size as a PKZIP-compatible compression engine. LZX compression requires more time, but LZX decompression is typically faster.

Examples:

```
.Set CompressionType=MSZIP  ; MSZIP compressor
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

DestinationDir=path
Path prefix to store in cabinet file for each file in the cabinet.

Default: .Set DestinationDir= ; Default is no path prefix

path is concatenated with a path separator ("\") and the target file name on File Copy Commands to produce the file name that is

stored in cabinet file. EXTRACT.EXE will use this file name as the default name when the file is extracted.

Examples:

```
.Set DestinationDir=SYSTEM  ; Following files get SYSTEM prefix
bin\ARIAL.TTF               ; Name in cabinet is SYSTEM\ARIAL.TTF
.Set DestinationDir=        ; No prefix
bin\ARIAL.TTF               ; Name in cabinet is ARIAL.TTF
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

DiskDirectoryn=directory

The output directory name for the specified disk.

Default: ; By default none of these variables are defined

If this variable is not defined for a particular disk, then MakeCAB uses the **DiskDirectoryTemplate** to construct the disk directory.

Examples:

```
.Set DiskDirectory1=disk.one
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

DiskDirectoryTemplate=template

Set the output directory name template. One directory is created for each disk of the layout.

Default: .Set DiskDirectoryTemplate=DISK* ; Default is DISK1, DISK2, etc.

As MakeCAB processes a directive file, it will create one or more disk "images". Rather than using some specific disk format, however, MakeCAB simply creates one subdirectory for each disk and places the files for each disk in the appropriate directory. If a "*" exists in this variable, then it is replaced with the disk number. If no "*" is specified, then all files are placed in the single directory specified by this variable.

This variable is used only if no variable **DiskDirectory*n*** exists for disk *n*.

NOTE:

Examples:

```
.Set DiskDirectoryTemplate=C:\EXCEL6\DISK*  ; Put files in separate dirs
.Set DiskDirectoryTemplate=C:\EXCEL6        ; Put all files in C:\EXCEL6
.Set DiskDirectoryTemplate=                 ; Put all files in current dir
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

DiskLabeln=label

The user-readable text string for the specified disk.

Default: ; By default none of these variables are defined

This *label* is stored in cabinet files that contain files that are split across disk boundaries, to simplify prompting for the appropriate disk to insert into the drive. For example, if EXCEL.EXE started in 1.CAB and finished in 2.CAB, and a user asked to extract EXCEL.EXE from 2.CAB, EXTRACT.EXE can retrieve the printed label for the disk containing 1.CAB (say, Excel Program Disk 1) and tell the user to insert that disk and try again.

If this variable is not defined for a particular disk, then MakeCAB uses the **DiskLabelTemplate** to construct the disk label.

Examples:

```
.Set DiskLabel1="Excel Setup Disk 1"
.Set DiskLabel2="Excel Setup Disk 2"
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

DiskLabelTemplate=template

Set the printed disk label. Used if individual **DiskLabel**n variables are not defined.

Default: .Set DiskLabelTemplate="Disk *" ; Default is "Disk 1", "Disk 2", etc.

Sets the default user-readable disk label. If a "*" exists in this variable, then it is replaced with the disk number. This variable is used only if no variable **DiskLabel*n*** exists for disk *n*.

Examples:

```
.Set DiskLabelTemplate="Excel Disk *"
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

DoNotCopyFiles=On | **Off**

Controls whether File Copy Commands actually copy files.

Default: .Set DoNotCopyFiles=Off ; Files *are* copied

This option is intended to be used when Cabinet is OFF and Compress is OFF, as a means of generating an INF file very quickly. It has no affect when Cabinet is ON or Compress is ON.

Examples:

```
.Set DoNotCopyFiles=ON      ; Make MakeCAB create the INF file quickly
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

FolderFileCountThreshold=count

Set the threshold on the number of files to store in a folder.

Default: .Set FolderFileCountThreshold=0 ; Default to no limit on count of files in a folder

Sets the threshold file count for the current folder. When this threshold is exceeded, then the current folder is closed. If any more files are to be processed, they will go into a new folder.

If **Cabinet** is **OFF**, this variable is ignored.

If *count* is 0, then there is no limit on the count of files in a folder.

Examples:

```
.Set FolderFileCountThreshold=50  ; No more than 50 files per folder
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

FolderSizeThreshold=size

Set the threshold size for the current folder.

Default: .Set MinMaxFolderSize=0 ; Default to the maximum cabinet size

Sets the threshold size for the current folder. When this threshold is exceeded, then the current folder is closed. If any more files are to be processed, they will go into a new folder. MakeCAB attempts to limit folders to the size specified by this variable, but in most cases folders will be a bit larger than this threshold.

If **Cabinet** is **OFF**, this variable is ignored. If *size* is 0, then the threshold is the same as the maximum cabinet size.

Folders are compression/encryption boundaries. The state of the compressor and cryptosystem are reset at folder boundaries. To access a file in a folder, the folder must be decrypted and decompressed starting from the **front** of the folder and continuing through to the desired file. Thus, smaller folder thresholds are appropriate for a layout where a small number of files needs to be randomly accessed quickly from a cabinet. On the other hand, larger folder thresholds permit the compressor to examine more data, and so generally yield better compression results. For a layout where the files will be accessed sequentially and most of the files will be accessed, a larger folder threshold is best.

Examples:

```
.Set FolderSizeThreshold=1M  ; Aim for 1Mb folders
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

GenerateInf=ON | OFF

Controls Unified vs. Relational INF generation mode.

Default: .Set GenerateInf=ON ; Default to "unified" INF mode

If **GenerateInf** is **ON** when the first file copy command is encountered, then Unified INF mode is selected. In this mode, file detail lines are written to the INF file as file copy commands are processed, so the order of file lines in the INF is exactly the same as the order of the files in the layout.

If **GenerateInf** is **OFF** when the first file copy command is encountered, then Relational INF mode is selected. In this mode, file copy commands are processed, but INF file generation is delayed until **GenerateInf** is set to **ON**, and File Reference commands are used to select information on files in the layout to be placed in the INF file.

Unified mode is easier to use, since each file is specified only once, and is most appropriate for quick usage of MakeCAB.

Relational mode is more complicated, since each file must be specified (at least) twice, but it provides very fine control of both the disk layout **and** the format of the INF file. In particular, some INF files want to have sections to list the files associated with a certain feature, there may be many such sections, and some files may be required in more than one section. Unified mode does not provide any method to generate such an INF file, but Relational mode does via the File Reference command.

By separating the disk layout order from the INF file order, MakeCAB permits optimization of the file layout for compression vs. access time. The *layout section* of the DDF contains file copy commands that control precisely where files are in the layout. The *INF section* of the DDF contains INF formatting information, including File Reference commands to pull in information about specific files from earlier File Copy commands in the layout section.

NOTE: Once **GenerateInf** is set to **ON** and at least one File Copy command has been processed, **GenerateInf** may not be set to **OFF** (i.e., in Relational Mode, all File Copy commands must be processed before any File Reference commands)

Examples:

```
;** Layout section - File Copy commands
.Set GenerateInf=OFF
foo.exe
bar.exe other.exe
foo.exe foo1.exe
```

```
....

;** INF section -- File Reference commands
.Set GenerateInf=ON
.WriteInf "[a section]"
foo.exe
other.exe
fool.exe /rename=sys\foo.exe   ; pass custom parameter
....
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

Inf*Xxx*=string

Sets the default value for an INF parameter.

Default: [Not applicable]

Variables of this form (other than the standard ones in this list) can be used for two purposes:

a. To override the usual value of a standard INF parameter (like **date**, **time**, **attr**, etc.) for all the files (or a set of files) in the layout.

b. To define a custom INF parameter, and specify its default value.

NOTE: When in Relation INF mode, only the **last value** for a particular **Inf***Xxx* variable will be carried over from the layout section to the INF section of the DDF. In the following example:

```
;** Layout section - File Copy commands
.Set GenerateInf=OFF    ; Select Relational INF
.Set InfCustom=apple
file.1
.Set InfCustom=pear
file.2
;** INF section - File Reference commands
.Set GenerateInf=ON
file.1                  ; *custom* value is "pear", not "apple"!
file.2
```

Examples:

```
.Set InfDate=05/02/94   ; Date stamp all files
.Set InfTime=06:00:00a  ; Time stamp all files
.Set InfAttr=           ; Turn off all attributes (esp. read-only)
.Set InfCustom=yes      ; Define custom INF parameter
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

InfCabinetHeader[*n*]=string

Sets the header text for the cabinet section of the INF file.

Default: .Set InfCabinetHeader="[cabinet list]"

This string is written to the INF prior to any cabinet detail lines. MakeCAB will also use any variables of the form **InfCabinetHeader***n* where *n* is an integer with no leading zeros (0). These additional lines will be printed out in increasing order after the **InfCabinetHeader** line. Any **.InfBegin Cabinet**/**.InfEnd** lines will be printed as they are encountered, but in any event after all of these header lines.

Examples:

```
.Set InfCabinetHeader=";Lots o' cabinets"

.Set InfCabinetHeader=                ; No cabinet header

.Set InfCabinetHeader=";Line 1 of cabinets"
.Set InfCabinetHeader1=";Line 2 of cabinets"
.Set InfCabinetHeader2=";Line 3 of cabinets"
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

InfCabinetLineFormat[*n*]=format string

Sets the detail line format for the cabinet section of the INF file.

Default: .Set InfCabinetLineFormat=*cab#*,*disk#*,*cabfile*

This format is used to generate a line in the "cabinet" section of the INF. If a numeric suffix *n* is specified in the variable name, then the specified format is used for cabinet number *n*. If no such cabinet number-specific format is defined, then the value of the **InfCabinetLineFormat** variable is used.

See InfDisk/Cabinet/FileLineFormat Syntax and Semantics for details on the format string.

See INF Parameters for a list of the allowed parameter names.

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

InfCommentString=string

Sets the line comment string for the INF file.

Default: .Set InfCommentString=";"

This is the string MakeCAB will use to prefix comment lines that it generates in the INF (the autogenerated MakeCAB version/date/time lines, for example).

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

InfDateFormat=YYYY-MM-DD | MM/DD/YY

Sets the date format used for dates written to the INF file.

Default: .Set InfDateFormat=MM/DD/YY ; Default to normal US convention

This format is used to format the **date** parameter for the **InfFileLineFormat** used to write file detail lines to the INF file.

Examples:

```
.Set InfDateFormat=YYYY-MM-DD       ; Use the preferred ACME format
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

InfDiskHeader[*n*]=string

Sets the header text for the disk section of the INF file.

Default: .Set InfDiskHeader="[disk list]"

This string is written to the INF prior to any disk detail lines. MakeCAB will also use any variables of the form **InfDiskHeader***n* where *n* is an integer with no leading zeros (0). These additional lines will be printed out in increasing order after the **InfDiskHeader** line. Any **.InfBegin Disk**/**.InfEnd** lines will be printed as they are encountered, but in any event after all of these header lines.

Examples:

```
.Set InfDiskHeader=";Lots o' Disks"

.Set InfDiskHeader=      ; No Disk header

.Set InfDiskHeader=";Line 1 of Disks"
.Set InfDiskHeader1=";Line 2 of Disks"
.Set InfDiskHeader2=";Line 3 of Disks"
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

InfDiskLineFormat[*n*]=format string

Sets the detail line format for the disk section of the INF file.

Default: .Set InfDiskLineFormat=*disk#*,*label*

This format is used to generate a line in the "disks" section of the INF. If a numeric suffix *n* is specified in the variable name, then the specified format is used for disk number *n*. If no such disk number-specific format is defined, then the value of the **InfDiskLineFormat** variable is used.

See InfDisk/Cabinet/FileLineFormat Syntax and Semantics for details on the format string.

See INF Parameters for a list of the allowed parameter names.

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

InfFileHeader[*n*]=string

Sets the header text for the file section of the INF file.

Default: .Set InfFileHeader="[file list]"

This string is written to the INF prior to any file detail lines. MakeCAB will also use any variables of the form **InfFileHeader***n* where *n* is an integer with no leading zeros (0). These additional lines will be printed out in increasing order after the **InfFileHeader** line. Any **.InfBegin File**/**.InfEnd** lines will be printed as they are encountered, but in any event after all of these header lines.

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

InfFileLineFormat[*n*]=format string

Sets the detail line format for the file section of the INF file.

Default: .Set InfFileLineFormat=*disk#*,*cab#*,*file*,*size*

This format is used to generate a line in the "file" section of the INF. If a numeric suffix *n* is specified in the variable name, then the specified format is used for file number *n* (file numbers start at 1, and are based on the File Copy Commands, not the File Reference Commands). If no such file number-specific format is defined, then the value of the **InfFileLineFormat** variable is used.

See InfDisk/Cabinet/FileLineFormat Syntax and Semantics for details on the format string.

See INF Parameters for a list of the allowed parameter names.

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

InfFileName=filename

Sets the name of the INF output file.

Default: .Set InfFileName=SETUP.INF ; Default file name is SETUP.INF

Defines the file name for the INF file. This file has disk, cabinet, and file information that is intended for use by a setup program during the setup process.

Examples:

```
.Set InfFileName=EXCEL.INF
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

InfFooter[*n*]=string

Sets the footer text for beginning of the INF file.

Default: // Run MakeCAB and use the .Dump command to see the default footer

These strings are written to the INF file after all other information. To disable this footer text, set InfFooter to the empty string (.Set InfFooter=). MakeCAB will also use any variables of the form **InfFooter***n* where *n* is an integer with no leading zeros (0). These additional lines will be printed out in increasing order after the **InfFooter** line, starting with **InfFooter1**.

The following special strings may be specified in InfFooter[n] values (note that the two percent signs are required, so that MakeCAB does not interpret these as variable references):

String	Description
%%1	The comment string -- each InfFooter[n] line should probably start with %%1.
%%2	The date and time MakeCAB was run to produce the INF file.
%%3	The version of MakeCAB use to produce the INF file.

Examples:

```
.Set InfFooter=             ; Disable INF footer text
.Set InfFooter="%%1 %2 %3"  ; Short footer
.Set InfFooter="%%1*****"   ; Long footer
.Set InfFooter1="%%1* %2"   ; Long footer continued
.Set InfFooter2="%%1* %3"   ; Long footer continued
.Set InfFooter3="%%1*****"  ; Long footer continued
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

InfHeader[*n*]=string

Sets the header text for beginning of the INF file.

Default: // Run MakeCAB and use the .Dump command to see the default header.

These strings are written to the INF file prior to any other information. To disable this header text, set InfHeader to the empty string (.Set InfHeader=). MakeCAB will also use any variables of the form **InfHeader***n* where *n* is an integer with no leading zeros (0). These additional lines will be printed out in increasing order after the **InfHeader** line, starting with **InfHeader1**.

The following special strings may be specified in InfHeader[n] values (note that the two percent signs are required, so that MakeCAB does not interpret these as variable references):

String	Description
%%1	The comment string -- each InfHeader[n] line should probably start with %%1.
%%2	The date and time MakeCAB was run to produce the INF file.
%%3	The version of MakeCAB use to produce the INF file.

Examples:

```
.Set InfHeader=             ; Disable INF header text
.Set InfHeader="%%1 %2 %3"  ; Short header
.Set InfHeader="%%1*****"   ; Long header
.Set InfHeader1="%%1* %2"   ; Long header continued
.Set InfHeader2="%%1* %3"   ; Long header continued
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

.Set InfHeader3="%%1***" ; Long header continuedInfSectionOrder=[D | C | F]***

Set the generation and relative order of the Disk, Cabinet, and File sections in the INF file.

Default: .Set InfSectionOrder=DCF ; Disk, then Cabinet, and then File

This variable controls what sections of the INF file are generated, and the order in which they appear. Each of the letters "C" (cabinet), "D" (disk), and "F" (file) may be used at most once. Any or all of these letters may be omitted, and the corresponding section of the INF file will not be generated.

Examples:

```
.Set InfSectionOrder=DF  ; Disks, then files, omit the cabinet section
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

MaxCabinetSize=size

Set the maximum size for the current cabinet.

Default: .Set MaxCabinetSize=0 ; No limit, except MaxDiskSize

size is the maximum size for the current cabinet. If **Cabinet** is **ON** when this maximum is exceeded, then the current folder being processed will be split between the current cabinet and the next cabinet. If **Cabinet** is **OFF**, then this variable is ignored.

Note that **MaxDiskSize** (or **MaxDiskSize***n*, if specified) takes precedence over this variable. MakeCAB never splits a cabinet file across a disk boundary, so a cabinet file will be no larger than the amount of free space available on the disk at the time the cabinet is created, even if this size is less than **MaxCabinetSize**.

If *size* is 0, then the cabinet size is limited only by the disk size (**MaxDiskSize or MaxDiskSize***n*).

Examples:

```
.Set MaxCabinetSize=0  ; Use disk size as limit
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

MaxDiskFileCount=count

Sets the maximum number of files that can be stored on a disk.

Default: .Set MaxDiskFileCount=0 ; Default is no limit

count is the maximum number of files to store on a disk. Once this count has been reached, MakeCAB will close the current disk, even if space remains on the disk. This variable is most useful when cabinet files are not being used (say, to simulate the old style setup where each file is individually compressed), and MakeCAB needs to understand the limit of the number of files that can be stored in the root directory of a floppy.

If *count* is 0, then there is no limit on the number files per disk.

You can use a standard disk size from the following list, and MakeCAB will supply the known FAT root directory limits for that disk size:

- **1.44M**
- **1.25M** (Japanese NEC 3.5" drive capacity)
- **1.2M**
- **720K**
- **360K**
- **CDROM**

The file count does **not** include any files inside cabinets. Each cabinet counts as a single file for purposes of this count.

Examples:

```
.Set DiskFileCountMax=256    ; Limit of 256 files per disk
.Set DiskFileCountMax=1.44M  ; Use limit for 1.44M FAT floppy disk
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

MaxDiskSize[*n*]=size

Set the maximum default size for a disk.

Default: .Set MaxDiskSize=1.44M ; Default is 1.44M floppy

size is the maximum default size for a disk. This variable is used only for disks for which a variable **MaxDiskSize***n* is not defined.

If **Cabinet** is **OFF**, and the next file to be laid out cannot fit on the current disk, then MakeCAB will move to the next disk. If **Cabinet** is **ON**, then the current cabinet will use as much space on the current disk as possible.

If *size* is 0, then the disk size is unlimited.

You can use a standard disk size from the following list, and MakeCAB will use the correct disk size, down to the byte:

- **1.44M**
- **1.25M** (Japanese NEC 3.5" drive capacity)
- **1.2M**
- **720K**
- **360K**
- **CDROM**

Examples:

```
.Set MaxDiskSize=0      ; No limit
.Set MaxDiskSize=CDROM  ; All files are being placed on a CD-ROM

.Set MaxDiskSize1=720K  ; First disk is 720K
.Set MaxDiskSize=1.44M  ; ... rest are 1.44M
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

MaxErrors=count

Set the maximum number of errors allowed before pass 1 terminates.

Default: .Set MaxErrors=20 ; Default is 20 errors

count is the maximum number of errors to permit before terminating pass 1.

If *count* is 0, then an unlimited number of errors is allowed.

Examples:

```
.Set MaxErrors=0  ; No limit
.Set MaxErrors=5  ; Limit to just a few
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

ReservePerCabinetSize=size

Sets a fixed size to reserve in a cabinet for the FCRESERVE structure.

Default: .Set ReservePerCabinetSize=0 ; Default is to reserve no space

size is the amount of space to reserve in a cabinet for the FCRESERVE structure. The total size of the FCRESERVE structure is the value of this variable plus the number of folders in the cabinet times the value of the **ReservePerFolderSize** variable.

size must be a multiple of 4 (to ensure memory alignment on certain systems).

A common use for this variable is to reserve space to store per-folder cryptosystem information, in the case where the cabinet is encrypted. For example, some sort of checksum value might be stored here to permit validation that the key being used to decrypt the cabinet is actually the one that was used to encrypt the cabinet.

MakeCAB fills this reserved section with zeros.

Examples:

```
.Set ReservePerCabinetSize=8  ; For use as a cryptosystem key checksum
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

ReservePerDataBlockSize=size

Sets the amount of space to reserve in each Data Block header.

Default: .Set ReservePerDataBlockSize=0 ; Default is to reserve no space

size is the amount of space to reserve in each Data Block header. This space is located after the standard Data Block header and before the data for the data block.

size must be a multiple of 4 (to ensure memory alignment on certain systems).

One possible use for this variable is to reserve space to store a per-data block cryptosystem information, in the case where the cabinet is encrypted. (See note below.)

NOTE: [6/6/94] Ali Baba is not using this value, so even though it has been implemented and tested, there are no known customers.

MakeCAB fills this reserved section with zeros.

Examples:

```
.Set ReservePerCabinetSize=4  ; Reserve 4 bytes per data block
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

ReservePerFolderSize=size

Sets the amount of additional space to reserve in the FCRESERVE structure for each folder in the cabinet.

Default: .Set ReservePerFolderSize=0 ; Default is to reserve no space

size is the amount of space to reserve in the FCRESERVE structure for each folder in the cabinet. The total size of the FCRESERVE structure is the value of this variable times the value of the number of folders in the cabinet, plus the value of the **ReservePerCabinetSize** variable.

size must be a multiple of 4 (to ensure memory alignment on certain systems).

A common use for this variable is to reserve space to store a per-folder cryptosystem key, in the case where the cabinet is encrypted.

MakeCAB fills this reserved section with zeros.

Examples:

```
.Set ReservePerCabinetSize=8  ; Size of an RC4 cryptosystem key
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

RptFileName=filename

Sets the name of the RPT output file.

Default: .Set RptFileName=SETUP.RPT ; Default file name is SETUP.RPT

Defines the file name for the RPT file. This file has summary information on the MakeCAB run.

Examples:

```
.Set RptFileName=EXCEL.RPT
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

SourceDir=path

The default path used to locate source files specified in File Copy Commands.

Default: .Set SourceDir= ; Default is to look in the current directory

path is concatenated with a path separator ("\") and the source file name on the File Copy Command to produce the file name used to find the source file.

If *path* is empty, then the source file name specified on the File Copy Command is not modified.

Examples:

```
.Set SourceDir=C:\PROJECT  ; Find all source files in c:\project
```

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax > Variable Details

UniqueFiles=ON | **OFF**

Controls whether destination file names in a layout must be unique.

Default: .Set UniqueFiles="ON" ; File names must be unique

If **UniqueFiles** is **ON**, MakeCAB checks that all destination file names (names stored on disks or in cabinets) are unique, and generates an error (during pass 1) if they are not. **ON** is the default, since using the same filename twice usually means that the same file was accidentally included twice, and this would be a waste of disk space.

If **UniqueFiles** is **OFF**, MakeCAB permits duplicate destination file names.

The **/UNIQUE** parameter may be specified on individual File Copy commands to override the value of **UniqueFiles**.

If the **GenerateInf** variable is used to select Relational INF generation, then **UniqueFiles** must always be **ON**, since MakeCAB uses the destination filename as the unique key to link File Reference commands back to File Copy commands.

Back to: Top of page > MAKECAB.EXE > MAKECAB.EXE Directive File Syntax

EXTRACT.EXE

Extract supports command-line extraction of files from cabinet files.

```
extract [/y] [/A] [/D | /E] [/L location] cabinet_file [file_spec ...]
extract [/y] compressed_file [destination_file]
```

Switches:

- **/A** Process **all** files in a cabinet set, starting with the *cabinet_file*.
- **/D** Only produce a directory listing (do not extract).
- **/E** Force extraction.
- **/L** Use the directory specified by *location*, instead of the current directory, as the default location to place extracted files.
- **/Y** Overwrite destination without prompting. The default is to prompt if the destination file already exists, and allow the customer to: a) overwrite the file, b) skip the file, c) overwrite this file and all subsequent files that may already exist, or d) exit.

Parameters:

compressed_file
: This is a cabinet file that contains a single file (example, FOO.EX_ containing FOO.EXE). If *destination_file* is not specified, then the file is extracted and given its original name in the current directory.

destination_file
: This can be either a relative path (".:, "..", "c:foo", etc.) or a fully qualified path, and may specify either a file (or files, if wild cards are included) or a directory. If a directory is specified, then the file name stored in the cabinet is used. Otherwise, *destination_file* is used as the complete file name for the extracted file.

cabinet_file
: This is a cabinet file that contains two or more files. If no *file_spec* parameter is specified, then a list of the files in the cabinet is displayed. If one or more *file_spec* parameters are specified, then these are used to select which files are to be extracted from the cabinet (or cabinets). Wild cards are allowed to specify multiple cabinets.

location
: Specifies the directory where extracted files should be placed.

file_spec
: Specifies files to be extracted from the cabinet(s). May contain ? and * wild cards. Multiple file_specs may be supplied.

Examples:

Command	Behavior
EXTRACT foo.ex_	Assuming foo.ex_ contained just the single file foo.exe, then foo.exe would be extracted and placed in the current directory.
EXTRACT foo.ex_ bar.exe	Assuming foo.ex_ contained just the single file foo.exe, then foo.exe would be extracted and placed in the current directory in the file bar.exe.
EXTRACT cabinet.1	Assuming cabinet.1 contains multiple files, then a list of the files stored in the cabinet would be displayed.
EXTRACT cabinet.1 *.exe	Extract all *.EXE files from cabinet.1 and place them in the current directory

Back to: Top of page

Microsoft MSZIP Data Compression Format

Copyright © 1997 Microsoft Corporation. All rights reserved.

Topics in this section

Introduction
Implementation Details
Where to Find the 'Deflate' Specifications

Introduction

This document describes the format of MSZIP compressed data as used in the MSZIP compression mode of Microsoft's cabinet files. The

purpose of this document is to allow anyone to encode or decode MSZIP compressed data.

Back to: Top of page

Implementation Details

MSZIP compression has only minor variations from Phil Katz's 'deflate' method. Rather than re-document this method, this document will explain these variations and refer the reader to publicly available 'deflate' documents. Some 'deflate' implementations may contain extensions to the original specifications, but MSZIP uses only the three basic modes of deflate: stored, fixed Huffman tree, and dynamic Huffman tree.

Each MSZIP data block is the result of a complete 'deflate' compression operation. Each block is flushed out of the compressor before the next block begins, so the last sub-block in each block will be marked as the 'end' of the stream. Any decoding trees are discarded after each block, with only the history buffer surviving from one block to the next. Each data block represents 32k uncompressed, except that the last block in a folder may be smaller. A two-byte MSZIP signature precedes the compressed encoding in each block, consisting of the bytes 0x43, 0x4B.

The maximum compressed size of each MSZIP block is 32k + 12 bytes. This allows for the data to be passed as two separate "stored" sub-blocks, which each have a 5-byte overhead, plus the 2-byte signature. The Microsoft MSZIP compressor will emit "stored" sub-blocks with a length of exactly 32k, while some implementations do not exceed 32k-1.

Whenever a cabinet folder boundary is reached, the compression history is discarded, so that decoding any folder does not require any prior data.

Back to: Top of page

Where to Find the "Deflate" Specifications

The "deflate" algorithm was original documented by Phil Katz in APPNOTE.TXT, which accompanied the PKZip software. Its most recent description can be found in RFC 1951. (Try ftp://ftp.uu.net/graphics/png/documents/zlib/zdoc-index.html for pointers to obtain this RFC.)

Back to: Top of page

Print E-Mail

How would you rate the quality of this content?

1 2 3 4 5 6 7 8 9

Poor ○ ○ ○ ○ ○ ○ ○ ○ ○ Outstanding

Tell us why you rated the content this way. (optional)

Be the first to rate this page!

Manage Your Profile | Legal | Contact Us | MSDN Flash Newsletter

© 2006 Microsoft Corporation. All rights reserved. Terms of Use | Trademarks | Privacy Statement

Microsoft